Database Management Systems (DBMS)

Project Report

ComplexityUniverse: AI-Powered Algorithmic Time & Space Complexity Analysis Platform

Student Name(s):	Animesh Gupta	(25U024044)
	Aditya Chaudhary	(25U024045)
	Ritik Raj	(25U024041)
	Gaurav Kumar Singh	(25U024043)
Branch & Section:	CSE - (Data Science)	
Course Instructor:	Dr. Priyankar Choudhary	
Academic Session:	2026–2027 (Monsoon Semester)	

Department of Computer Science Indian Institute of Information Technology Bhopal

Contents

General Instructions **4**

1 Introduction **5**
1.1 Project Overview 5
1.2 Problem Statement 5
1.3 Objectives 6
1.4 Scope of the Project 6
1.5 Software and Hardware Requirements 7
1.5.1 Software Requirements 7
1.5.2 Hardware Requirements 7

2 System Requirements and Analysis **8**
2.1 Functional Requirements 8
2.2 Business Rules 9

3 Entity–Relationship (ER) Modeling **10**
3.1 Identification of Entities 10
3.2 Identification of Relationships 11
3.3 ER Diagram 11
3.3.1 Type 1: Conceptual ER Modeling (Peter Chen Notation) 11
3.3.2 Type 2: Logical / Relational ER Modeling (Crow's Foot Notation) 14
3.4 Explanation of the ER Diagram 17

4 ER-to-Relational Mapping **18**
4.1 Mapping Rules 18
4.2 Step-by-Step Conversion 18
4.3 Final Relational Schema 19

5 Functional Dependencies **20**
5.1 Identification of Functional Dependencies 20
5.2 Candidate Keys and Attribute Classification 21
5.2.1 Attribute Closure Proofs 21
5.3 FD Analysis 21

6 Database Normalization **23**
6.1 Unnormalized Form (UNF) . . . 23
6.2 First Normal Form (1NF) . . . 23
6.3 Second Normal Form (2NF) . . . 23
6.4 Third Normal Form (3NF) . . . 24
6.5 Boyce–Codd Normal Form (BCNF) . . . 24
6.6 Final Normalized Schema . . . 24

7 Relational Integrity and Constraints **25**

8 MySQL Database Implementation **27**
8.1 Database Creation . . . 27
8.2 Table Creation . . . 27
8.3 Data Insertion . . . 29
8.4 Data Modification and Deletion . . . 29
8.5 Sample Database Contents . . . 29
8.6 SQL Query Implementation . . . 29
8.7 Advanced DBMS Features . . . 32
8.7.1 Database Views . . . 32
8.7.2 Database Triggers . . . 32
8.7.3 Stored Generated Columns . . . 33
8.7.4 ACID Transactions . . . 33

9 Application Development **34**
9.1 System Architecture . . . 34
9.2 Front-End Technologies . . . 34
9.3 Application Modules . . . 34
9.4 Database Connectivity . . . 35
9.5 Application Screenshots . . . 35
9.5.1 Algorithmic Code Editor and Analysis Submission . . . 35
9.5.2 Asymptotic Complexity Breakdown and Performance Profiling . . . 35
9.5.3 Analyzed Source Code Inspector and Platform Architecture Pillars 38
9.5.4 Interactive Curriculum and Algorithmic Knowledge Base . . . 38
9.5.5 Developer Dashboard: Saved Analyses History and Metrics . . . 40
9.5.6 Developer Dashboard: Profile Metadata and Real-Time Audit Feed 40
9.5.7 Administrator Console: Platform Overview and Distribution Telemetry . . . 42
9.5.8 Administrator Console: AI Governance and Prompt Engineering . 42
9.6 Database Operations through the Application . . . 44

10 Testing and Results **45**
10.1 Test Cases . . . 45
10.2 Query Testing . . . 46

10.3 Constraint Testing 46
10.4 Results and Discussion 46

11 Conclusion and Future Scope **48**
11.1 Conclusion 48
11.2 Limitations 48
11.3 Future Scope 48

References **50**

A Complete MySQL Script **51**

B Source Code **52**

C Additional Screenshots **53**
C.1 User Authentication and Access Control Interfaces 53
C.2 Developer Onboarding and Knowledge Reinforcement 55
C.3 Content Management and User Administration 58

D Team Contribution **60**

Suggested Evaluation Scheme **62**

General Instructions

This report fulfills all requirements prescribed for the Database Management Systems (DBMS) laboratory curriculum at IIIT Bhopal:

1. **Real-World Application Selection:** ComplexityUniverse is an engineering platform for static code analysis, asymptotic complexity calculation (Big-O notation), and algorithmic learning.
2. **Mandatory MySQL Implementation:** The backend strictly uses **MySQL 8.0** on Aiven Cloud with InnoDB, utf8mb4 encoding, foreign keys with cascade constraints, triggers, views, and SSL/TLS certificate authentication (`ca.pem`).
3. **Design Preceding Implementation:** Conceptual ER modeling (Chen notation) and Relational modeling (Crow's Foot notation) were designed prior to full-stack implementation.
4. **Demonstration of Core DBMS Concepts:**
   - Conceptual and Relational ER Modeling;
   - Formal ER-to-Relational schema mapping;
   - Functional Dependencies (FDs), Armstrong's Axioms, and Attribute Closure (X^+);
   - Candidate key derivations, minimal cover, and lossless-join verification;
   - Normalization through 1NF, 2NF, 3NF, and BCNF;
   - Relational integrity constraints (PK, FK, UNIQUE, NOT NULL, CHECK, CASCADE);
   - SQL DDL, DML, sample data, and 10 categories of SQL queries;
   - Advanced features: 5 Views, 10 Triggers, Stored Generated Columns, and ACID Transactions.
5. **Deliverables:** Complete LaTeX documentation, application source code, and database scripts (`schema.sql` and `seed.sql`).

Note: ComplexityUniverse: AI-Powered Algorithmic Time & Space Complexity Analysis Platform replaces the default inventory example while strictly maintaining the required chapter sequence and section guidelines.

Chapter 1

Introduction

1.1 Project Overview

ComplexityUniverse is a web platform designed to analyze, visualize, and explain the asymptotic Time and Space complexity (Big-O notation) of computer source code. In computer science and software development, understanding algorithmic efficiency with respect to input size n is essential for writing scalable software.

Traditional static linters only inspect syntax and formatting rules, while dynamic profilers measure wall-clock runtimes on fixed hardware without establishing general mathematical bounds. ComplexityUniverse bridges this gap by pairing Google Gemini AI semantic reasoning with a BCNF-normalized MySQL 8.0 relational database. The platform delivers:

- **Asymptotic Big-O Badging:** Worst, average, and best-case classifications: $\mathcal{O}(1)$, $\mathcal{O}(\log n)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$, and $\mathcal{O}(2^n)$.
- **Line-by-Line Cost Breakdown:** Clear identification of loop bounds, recursion depth, and auxiliary memory.
- **Actionable Optimization Tips:** Practical guidance for reducing asymptotic bounds.
- **Learn Curriculum Library:** Curated algorithmic topics with runnable code examples and full-text search.
- **Developer Dashboard:** Persistent historical analyses, saved bookmarks, and complexity telemetry.
- **Admin AI Governance Studio:** In-database prompt management with automatic trigger-based version auditing.

1.2 Problem Statement

According to Rice's Theorem and the Halting Problem, no general deterministic algorithm can decide non-trivial semantic properties of arbitrary code. Standard compiler AST

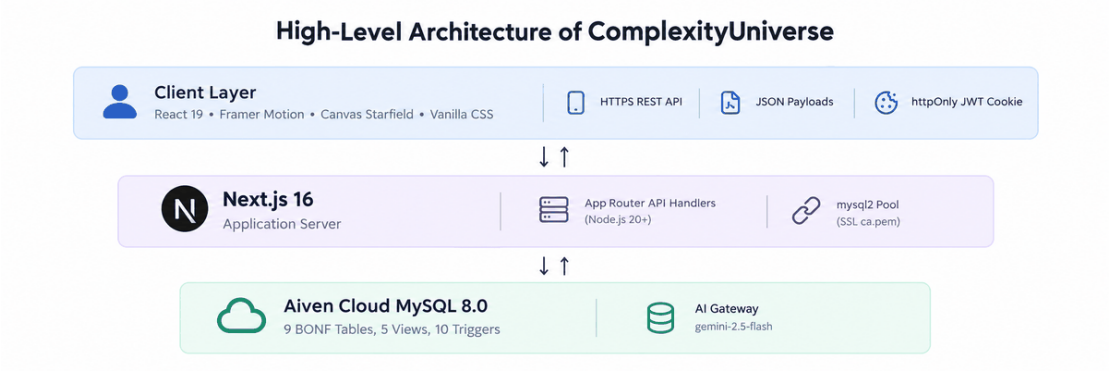


Fig. 1.1: Full-Stack Architecture of ComplexityUniverse

parsers struggle with dynamic loop termination, recursion trees, and complex standard library functions. At the same time, available developer tools lack persistent relational storage, structured educational tracks, and accessible student workflows. ComplexityUniverse provides a unified platform to analyze, store, and study algorithmic complexity.

1.3 Objectives

The primary objectives of the project are:

1. **Automate Complexity Evaluation:** Integrate Google Gemini with structured JSON output enforcement to evaluate code Time and Space bounds.
2. **Provide Granular Code Deconstruction:** Generate line-by-line cost attributions and optimization tips.
3. **Architect a BCNF Relational Schema:** Design a 9-table MySQL database strictly adhering to Boyce–Codd Normal Form.
4. **Implement Autonomous Business Invariants:** Use MySQL triggers to maintain counters, sanitize inputs, enforce versioning, and log audit events.
5. **Deliver High-Speed Analytics via Views:** Construct 5 pre-compiled views to optimize dashboard queries.
6. **Integrate Stored Generated Columns:** Compute physical line metrics automatically at the storage engine level.
7. **Build a Modern Responsive Interface:** Develop a full-stack Next.js 16 / React 19 web application with Vanilla CSS tokens and canvas particle graphics.
8. **Ensure Security & ACID Transactions:** Enforce SSL database transport encryption, bcrypt hashing, stateless JWT cookies, and ACID transaction boundaries.

1.4 Scope of the Project

The platform supports multi-language code analysis (JavaScript, TypeScript, Python, Java, C, C++, Go, Rust), structured Big-O reporting, user workspaces with persis-

tent histories, a searchable curriculum library, personal bookmarking with notes, and an administrative governance studio. The system operates via static semantic reasoning, avoiding remote code execution risks.

1.5 Software and Hardware Requirements

1.5.1 Software Requirements

- **Operating System:** Windows 11, Linux (Ubuntu 22.04+), or macOS Sonoma.
- **DBMS:** MySQL 8.0 on Aiven Cloud with InnoDB, utf8mb4 encoding, and TLS/SSL encryption (`ca.pem`).
- **Web Framework:** Next.js 16 (App Router, Turbopack) on Node.js 20+ LTS.
- **Frontend:** React 19, Framer Motion, Vanilla CSS (design tokens in `globals.css`).
- **Libraries:** `mysql2/promise`, `@google/genai`, `bcryptjs`, `jsonwebtoken`, `lucide-react`.

1.5.2 Hardware Requirements

- **Client:** Modern browser, 4 GB RAM minimum (8 GB recommended), 1080p display.
- **Server / Database Host:** Cloud container with 1 vCPU, 2 GB RAM, 10 GB SSD, and stable Internet connectivity with SSL support.

Chapter 2

System Requirements and Analysis

2.1 Functional Requirements

The functional requirements describe the core operational capabilities of ComplexityUniverse:

FR-1. User Management: Support user registration, bcrypt password hashing, stateless JWT authentication via secure cookies, and profile customization (headline, bio, location, avatar) (see Fig. C.2, Fig. C.1, and Fig. 9.6).

FR-2. Code Analysis Engine: Accept source code in multiple languages, send structured prompts to Gemini AI, and return worst/average/best-case Big-O bounds, line-by-line costs, and optimization advice (see Fig. 9.1 and Fig. 9.2).

FR-3. Storage Engine Metrics: Automatically calculate code line counts via MySQL stored generated columns upon tuple insertion (monitored in the dashboard telemetry cards, Fig. 9.5).

FR-4. Analysis History: Allow users to view, search, paginate, and delete saved analyses; triggers automatically update user analysis counters and insert audit records (see Fig. 9.5).

FR-5. Curriculum Library: Provide structured topic modules across major algorithmic paradigms with full-text search capability (see Fig. 9.4).

FR-6. Topic Bookmarking: Allow users to bookmark curriculum topics with personal study notes; reject duplicate bookmarks (see Fig. C.6).

FR-7. Admin Studio: Allow administrators to review platform metrics, edit AI prompt templates (with automatic version incrementation), and configure the active AI model (see Fig. 9.7, Fig. C.7, and Fig. 9.8).

FR-8. Audit Logging: Record user registrations, analysis creation/deletion, and bookmark updates into an append-only audit table (streamed in the audit panels of Fig. 9.6 and Fig. 9.7).

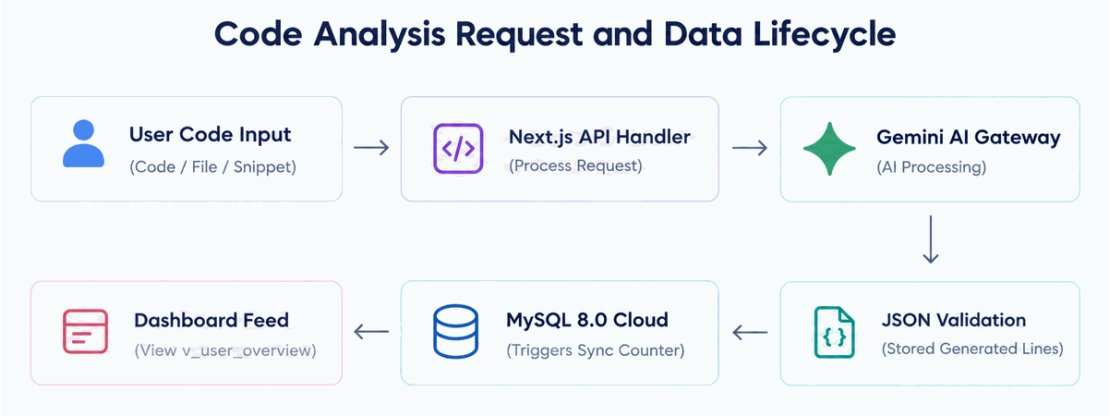


Fig. 2.1: Algorithmic Analysis Execution Lifecycle

2.2 Business Rules

The operational logic and database constraints are enforced by ten business rules:

BR-1. Unique Email: Every user email must be unique, lowercased, and whitespace-trimmed.

BR-2. Mandatory 1:1 Profile: Every user must have exactly one profile record, auto-created upon registration via trigger `trg_users_after_insert`.

BR-3. Cascading Deletion: Deleting a user must atomically remove their profile, saved analyses, bookmarks, and activity logs via `ON DELETE CASCADE`.

BR-4. Synchronized Counter: `users.analysis_count` must accurately mirror saved analyses, incrementing on insert and decrementing on delete via triggers.

BR-5. Unique Topic Slug: Every educational topic must have a unique URL slug, generated automatically from the title if omitted.

BR-6. Bookmark Uniqueness: A user cannot bookmark the same topic more than once, enforced by a composite unique key on `(user_id, topic_id)`.

BR-7. Role-Based Access: Admin routes and prompt mutations are restricted to accounts where `role = 'admin'`.

BR-8. Prompt Version Auditing: Updating an AI prompt template must automatically increment `version = version + 1` and refresh `updated_at` via a trigger.

BR-9. Immutable Line Count: `code_lines` in `code_analyses` must be computed solely by the storage engine using a stored generated expression.

BR-10. Session Security: API operations modifying user data must verify JWT token identity against the target resource's `user_id`.

Chapter 3

Entity–Relationship (ER) Modeling

3.1 Identification of Entities

The ComplexityUniverse data model comprises 9 entities:

1. **`users` (Strong Entity):** Stores user accounts. Attributes: `id` (PK), `name`, `email` (Unique), `password_hash`, `last_login_at`, `role`, `status`, `analysis_count` (Derived), `created_at`, `updated_at`.
2. **`user_profiles` (1:1 Strong Entity):** User profile details. Attributes: `id` (PK), `user_id` (FK, Unique), `headline`, `bio`, `location`, `avatar_seed`.
3. **`analysis_prompts` (Strong Entity):** Versioned AI prompts. Attributes: `id` (PK), `name`, `prompt_template`, `is_active`, `version`, `updated_by` (FK), `created_at`, `updated_at`.
4. **`code_analyses` (Strong Entity):** Saved analysis reports. Attributes: `id` (PK), `user_id` (FK), `title`, `language`, `code_text`, `code_lines` (Stored Generated), `time_complexity`, `space_complexity`, `summary`, `detailed_analysis`, `detail_mode`, `engine`, `prompt_version`, `created_at`, `updated_at`.
5. **`complexity_topics` (Strong Entity):** Curriculum topics. Attributes: `id` (PK), `topic_name`, `slug` (Unique), `category`, `difficulty`, `time_complexity`, `space_complexity`, `summary`, `notes_html`, `sort_order`, `is_published`, `created_at`, `updated_at`.
6. **`topic_examples` (Strong Entity):** Runnable code examples. Attributes: `id` (PK), `topic_id` (FK), `title`, `language`, `code_text`, `analysis_html`, `time_complexity`, `space_complexity`, `sort_order`, `created_at`.
7. **`user_saved_topics` (Associative Entity):** Resolves the $M : N$ bookmarking relationship. Attributes: `id` (PK), `user_id` (FK), `topic_id` (FK), `note`, `saved_at`. Composite Key: `(user_id, topic_id)`.
8. **`activity_log` (Strong Entity):** System audit records. Attributes: `id` (PK), `user_id` (FK, Nullable), `action`, `entity`, `entity_id`, `detail`, `created_at`.
9. **`app_settings` (Strong Entity):** Key-value configuration. Attributes: `setting_key` (PK), `setting_value`, `updated_at`.

3.2 Identification of Relationships

Table 3.1 summarizes the semantic relationships and structural constraints.

Relationship	Entities	Ratio	Participation	Action
Has_Profile	users ↔ user_profiles	1 : 1	Total (Profile), Partial (User)	CASCADE
Performs_Analysis	users ↔ code_analyses	1 : N	Total (Analysis), Partial (User)	CASCADE
Bookmarks	users ↔ complexity_topics	M : N	Partial (Both)	CASCADE
Contains_Examples	complexity_topics ↔ topic_examples	1 : N	Total (Example), Partial (Topic)	CASCADE
Manages_Prompts	users ↔ analysis_prompts	1 : N	Partial (Both)	SET NULL
Generates_Log	users ↔ activity_log	1 : N	Total (Log), Partial (User)	CASCADE

Table 3.1: Entity Relationship Identification and Constraints

3.3 ER Diagram

The database is modeled at two distinct abstraction tiers:

1. **Type 1: Conceptual ER Modeling (Peter Chen Notation):** Captures business entities, relationships, attributes, and derived attributes (dashed ellipses for `analysis_count` and `code_lines`).
2. **Type 2: Relational ER Modeling (Crow's Foot Notation):** Illustrates physical relational schemas, explicit column types, primary keys, foreign keys, and unique constraints across modular subsystems.

3.3.1 Type 1: Conceptual ER Modeling (Peter Chen Notation)

Fig. 1.1 (Fig. 1.1) presents the complete enterprise conceptual diagram, followed by the modular subsystem diagrams (Fig. 1.2 through Fig. 1.5):

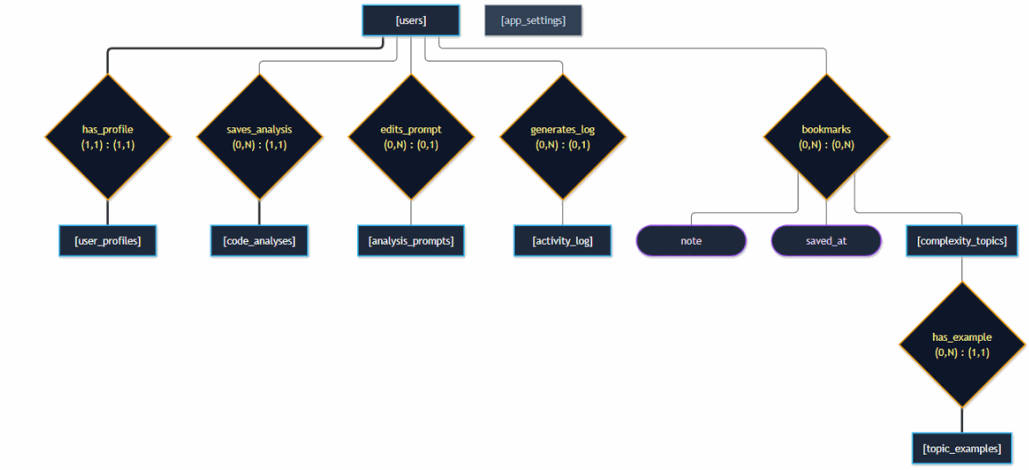


Fig. 1.1: Enterprise Conceptual ER Diagram (Peter Chen Notation)

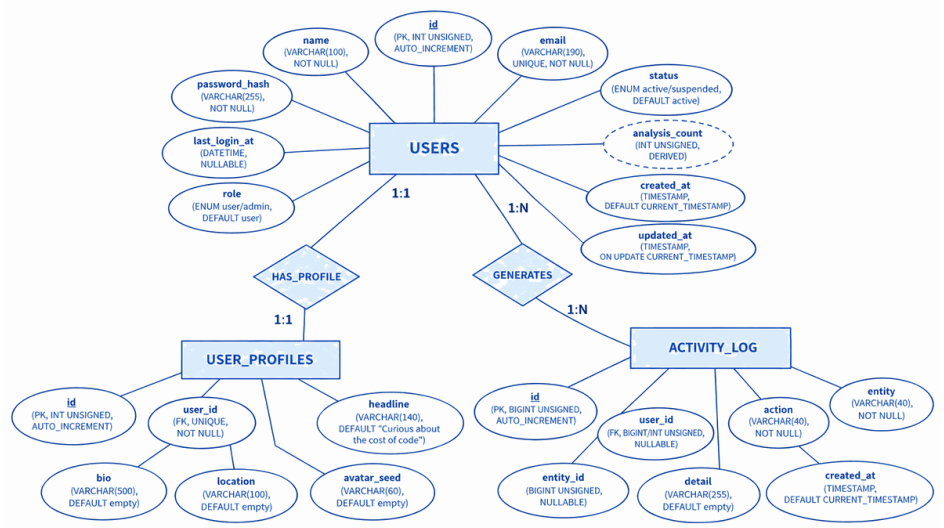


Fig. 1.2: Conceptual Subsystem 1: User Identity, Profile & Activity Log

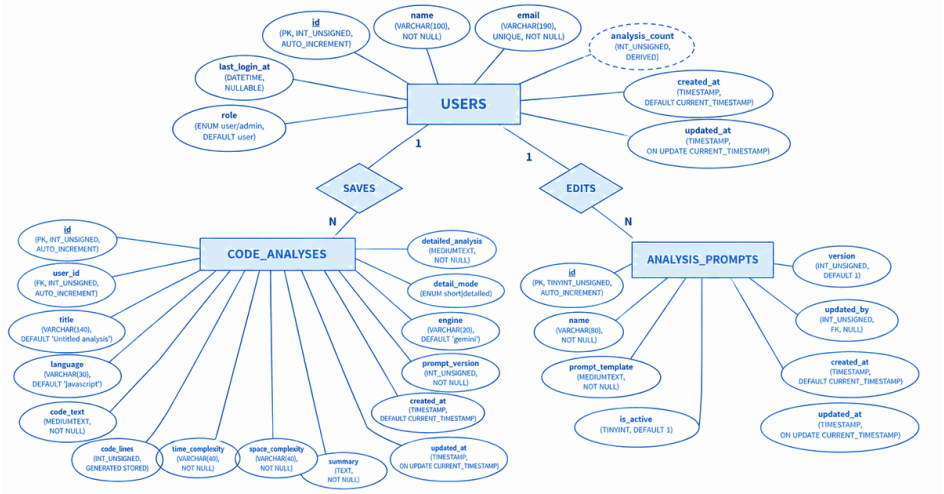


Fig. 1.3: Conceptual Subsystem 2: Code Analysis Engine & AI Prompts

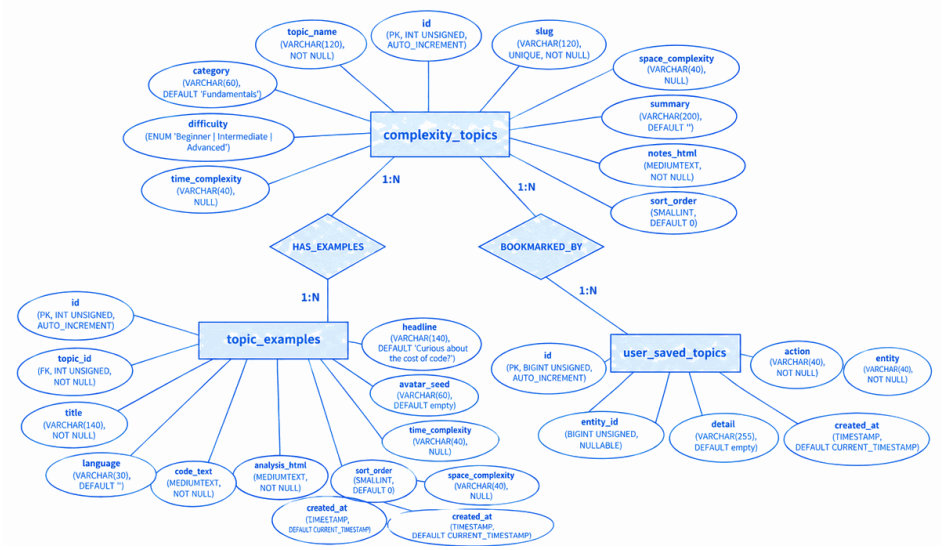


Fig. 1.4: Conceptual Subsystem 3: Curriculum Topics & Bookmarks

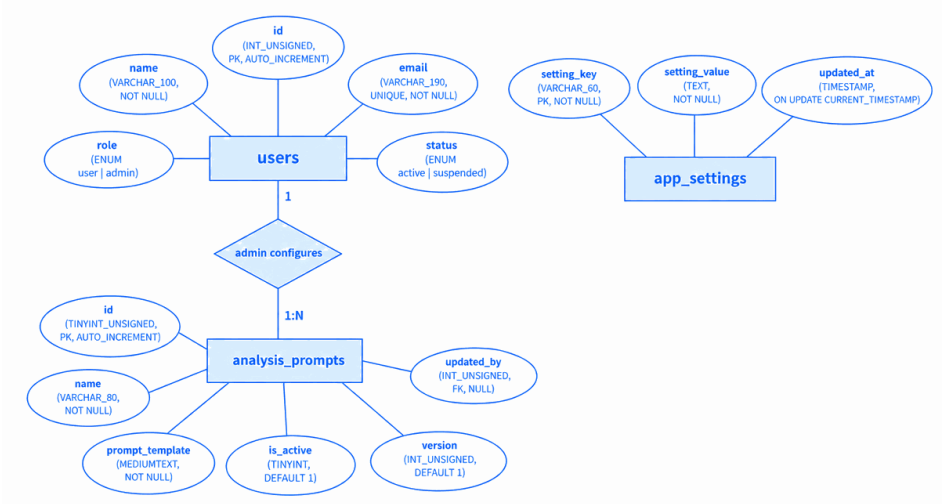


Fig. 1.5: Conceptual Subsystem 4: System Administration & Runtime Configuration

3.3.2 Type 2: Logical / Relational ER Modeling (Crow's Foot Notation)

The physical relational schemas implemented in MySQL 8.0 on Aiven Cloud are organized into four modular subsystem diagrams (Fig. 2.2 through Fig. 2.5) defining exact column data types, key constraints, and foreign key references:

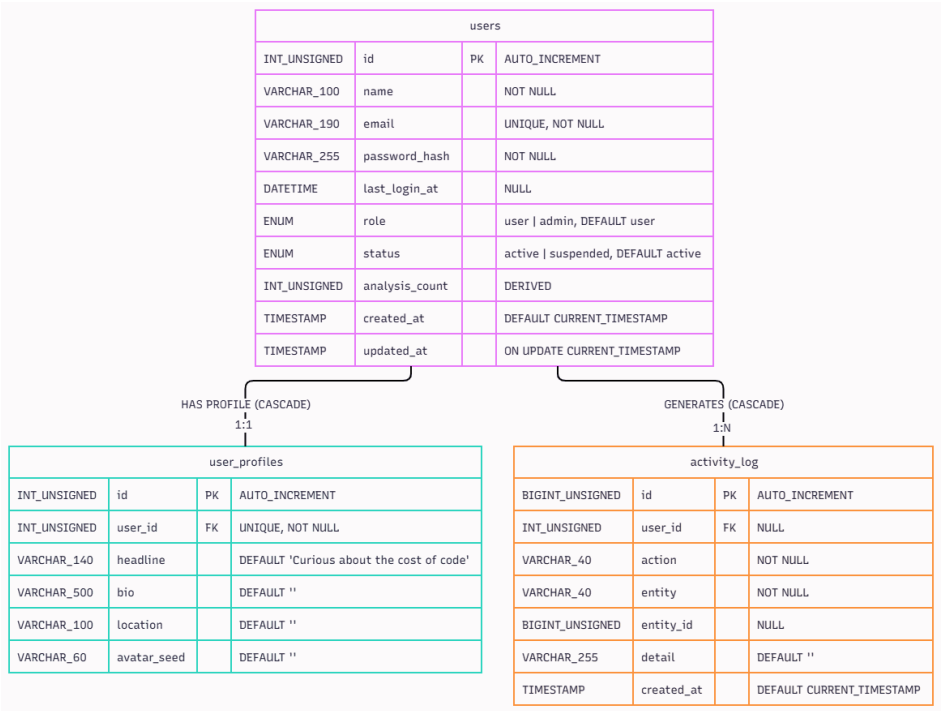


Fig. 2.2: Relational Subsystem 1: User Identity, Profile & Audit Schema

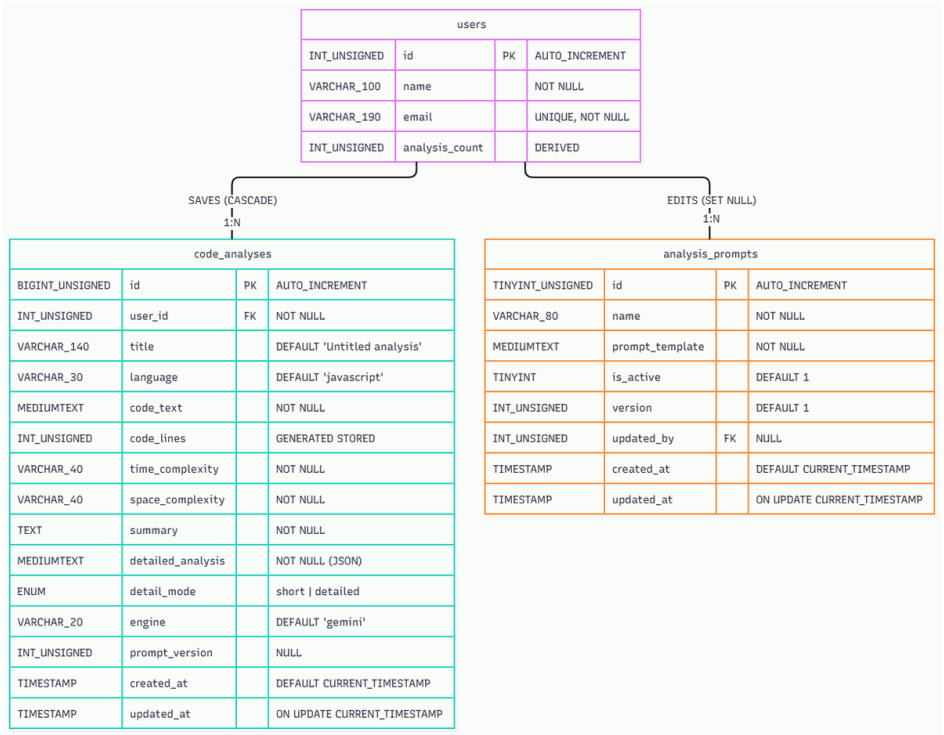


Fig. 2.3: Relational Subsystem 2: Code Analysis Engine & AI Prompts Schema

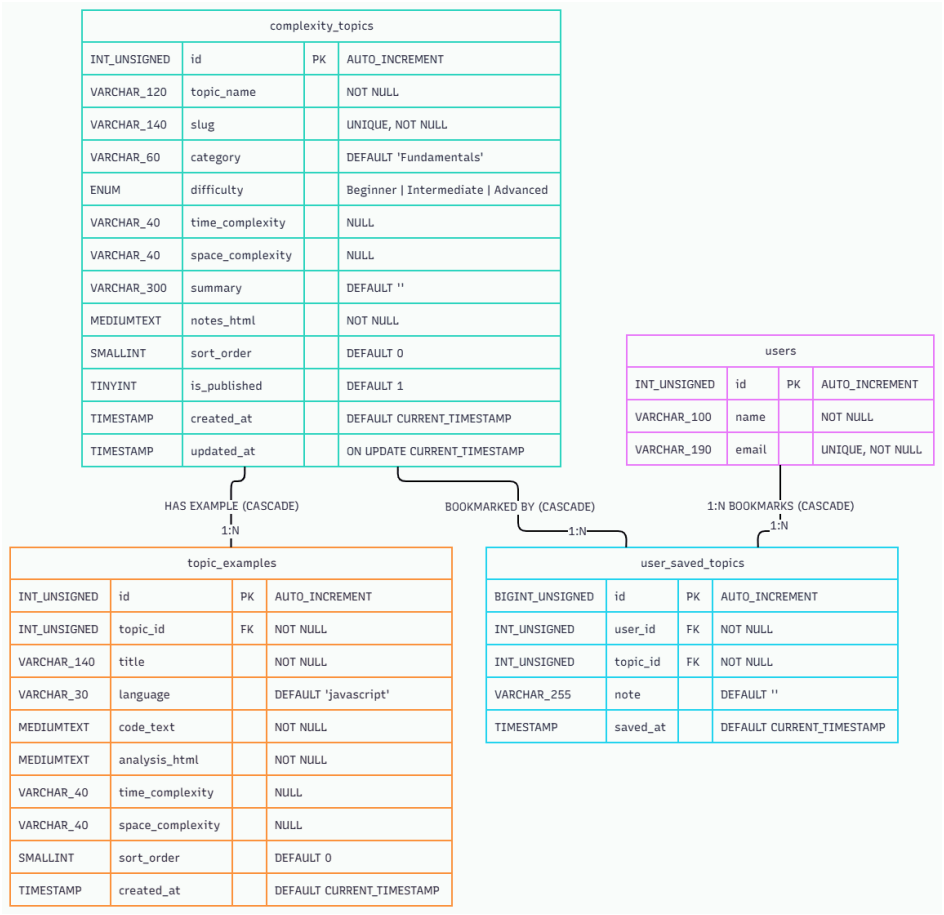


Fig. 2.4: Relational Subsystem 3: Educational Library & Bookmarks Schema

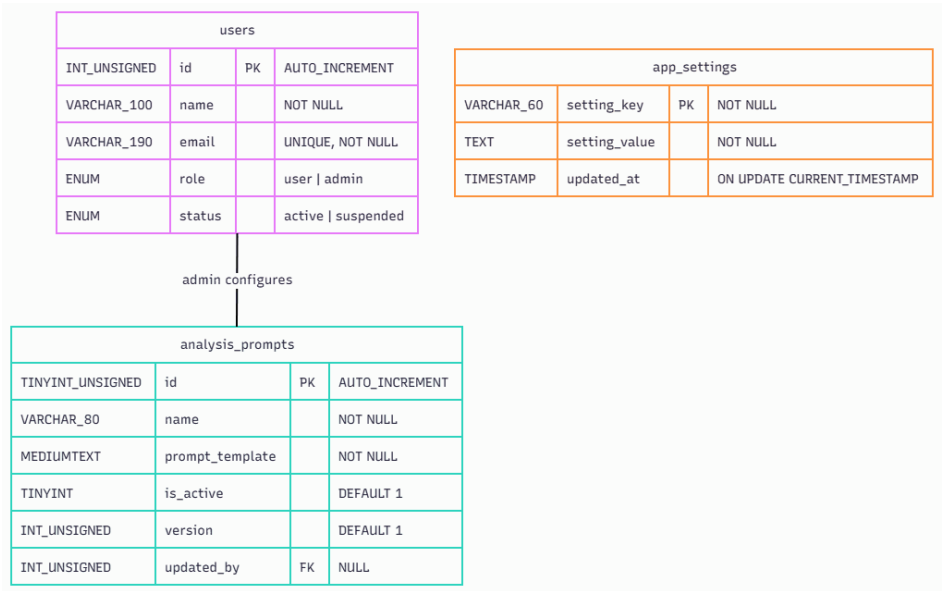


Fig. 2.5: Relational Subsystem 4: System Administration & Runtime Configuration Schema

3.4 Explanation of the ER Diagram

The diagrams illustrate the structure and constraints of the database:

- **Conceptual vs. Relational Modeling:** Conceptual diagrams (Fig. 1.1 to 1.5) capture business entities, relationships, and derived attributes (`analysis_count`, `code_lines` in dashed ellipses) without physical keys. The relational diagrams (Fig. 2.2 to 2.5) materialize primary keys, foreign keys, physical column types, and cascade rules.
- **1:1 Relationship (`users` to `user_profiles`):** Each user has at most one profile, and each profile belongs to exactly one user. Total participation on `user_profiles` is enforced by the unique foreign key `user_id` with cascade deletion.
- **1:N Relationship (`users` to `code_analyses`):** A user can save multiple analyses $(0..N)$. Each analysis record belongs to one user (1..1) with cascade deletion.
- **M:N Relationship Resolution (`users` to `complexity_topics`):** Modeled as a direct relationship in Chen notation and mapped into the associative table `user_saved_topics` with two $1 : N$ foreign keys and a composite unique constraint on (`user_id`, `topic_id`).
- **1:N Relationship (`complexity_topics` to `topic_examples`):** Each curriculum topic includes multiple code examples $(0..N)$ with total participation on the examples side.

Chapter 4

ER-to-Relational Mapping

4.1 Mapping Rules

The ER model is converted to relational schemas using standard database mapping rules:

1. **Rule 1 (Strong Entities):** For each strong entity set, create a relation including all simple attributes. Designate a primary key (PK).
2. **Rule 2 (Weak Entities):** Include owner entity primary key as a foreign key (FK), with composite primary key where applicable.
3. **Rule 3 (Binary 1:1 Relationships):** Include the PK of one entity as an FK in the other (choosing the entity with total participation). Enforce a `UNIQUE` constraint on the FK column.
4. **Rule 4 (Binary 1:N Relationships):** Place the primary key of the "1" side as a foreign key in the relation on the "N" side.
5. **Rule 5 (Binary M:N Relationships):** Create an associative relation containing the primary keys of both participating entities as foreign keys, plus any relationship attributes.
6. **Rule 6 (Multivalued Attributes):** Create a separate relation with the attribute and the parent entity's primary key as an FK.

4.2 Step-by-Step Conversion

Applying these rules systematically yields the physical tables:

1. **Step 1: Strong Entities:**
   - `users` maps to **USERS** (`id`, `name`, `email`, ...) with primary key `id`.
   - `complexity_topics` maps to **COMPLEXITY_TOPICS** (`id`, `topic_name`, `slug`, ...) with primary key `id`.
   - `app_settings` maps to **APP_SETTINGS** (`setting_key`, `setting_value`, `updated_at`) with primary key `setting_key`.

2. **Step 2: Binary 1:1 Relationship (`Has_Profile`):** Relation **USER_PROFILES** (id, user_id, ...) includes foreign key user_id → users(id) with constraint UNIQUE(user_id) and ON DELETE CASCADE.
3. **Step 3: Binary 1:N Relationships:**
   - **CODE_ANALYSES** includes foreign key user_id → users(id) with ON DELETE CASCADE.
   - **TOPIC_EXAMPLES** includes foreign key topic_id → complexity_topics(id) with ON DELETE CASCADE.
   - **ACTIVITY_LOG** includes foreign key user_id → users(id) with ON DELETE CASCADE.
   - **ANALYSIS_PROMPTS** includes foreign key updated_by → users(id) with ON DELETE SET NULL.
4. **Step 4: Binary M:N Relationship (`Bookmarks`):** Associative relation **USER_SAVED_TOPICS** (id, user_id, topic_id, note, saved_at) with composite candidate key on (user_id, topic_id) and cascade delete rules.

4.3 Final Relational Schema

The complete relational schema:

1. **USERS** (id, name, email, password_hash, last_login_at, role, status, analysis_count, created_at, updated_at)
2. **USER_PROFILES** (id, user_id, headline, bio, location, avatar_seed)
3. **ANALYSIS_PROMPTS** (id, name, prompt_template, is_active, version, updated_by, created_at, updated_at)
4. **CODE_ANALYSES** (id, user_id, title, language, code_text, code_lines, time_complexity, space_complexity, summary, detailed_analysis, detail_mode, engine, prompt_version, created_at, updated_at)
5. **COMPLEXITY_TOPICS** (id, topic_name, slug, category, difficulty, time_complexity, space_complexity, summary, notes_html, sort_order, is_published, created_at, updated_at)
6. **TOPIC_EXAMPLES** (id, topic_id, title, language, code_text, analysis_html, time_complexity, space_complexity, sort_order, created_at)
7. **USER_SAVED_TOPICS** (id, user_id, topic_id, note, saved_at)
8. **ACTIVITY_LOG** (id, user_id, action, entity, entity_id, detail, created_at)
9. **APP_SETTINGS** (setting_key, setting_value, updated_at)

Chapter 5

Functional Dependencies

5.1 Identification of Functional Dependencies

A functional dependency $X \to Y$ holds on R if tuples agreeing on X must also agree on Y. The non-trivial dependencies across the 9 relations are:

1. **users:**
   - FD_1: `id` $\to$ {`name`, `email`, `password_hash`, `last_login_at`, `role`, `status`, `analysis_count`, `created_at`, `updated_at`}
   - FD_2: `email` $\to$ {`id`, `name`, `password_hash`, `last_login_at`, `role`, `status`, `analysis_count`, `created_at`, `updated_at`}
2. **user_profiles:**
   - FD_1: `id` $\to$ {`user_id`, `headline`, `bio`, `location`, `avatar_seed`}
   - FD_2: `user_id` $\to$ {`id`, `headline`, `bio`, `location`, `avatar_seed`}
3. **analysis_prompts:**
   - FD_1: `id` $\to$ {`name`, `prompt_template`, `is_active`, `version`, `updated_by`, `created_at`, `updated_at`}
4. **code_analyses:**
   - FD_1: `id` $\to$ {`user_id`, `title`, `language`, `code_text`, `code_lines`, `time_complexity`, `space_complexity`, `summary`, `detailed_analysis`, `detail_mode`, `engine`, `prompt_version`, `created_at`, `updated_at`}
5. **complexity_topics:**
   - FD_1: `id` $\to$ {`topic_name`, `slug`, `category`, `difficulty`, `time_complexity`, `space_complexity`, `summary`, `notes_html`, `sort_order`, `is_published`, `created_at`, `updated_at`}
   - FD_2: `slug` $\to$ {`id`, `topic_name`, `category`, `difficulty`, `time_complexity`, `space_complexity`, `summary`, `notes_html`, `sort_order`, `is_published`, `created_at`, `updated_at`}

6. **topic_examples:**
   - FD_1: id $\rightarrow$ {topic_id, title, language, code_text, analysis_html, time_complexity, space_complexity, sort_order, created_at}
7. **user_saved_topics:**
   - FD_1: id $\rightarrow$ {user_id, topic_id, note, saved_at}
   - FD_2: {user_id, topic_id} $\rightarrow$ {id, note, saved_at}
8. **activity_log:**
   - FD_1: id $\rightarrow$ {user_id, action, entity, entity_id, detail, created_at}
9. **app_settings:**
   - FD_1: setting_key $\rightarrow$ {setting_value, updated_at}

5.2 Candidate Keys and Attribute Classification

A candidate key K satisfies $K^+ = R$ (sufficiency) and no proper subset determines R (minimality). Table 5.1 summarizes keys and prime/non-prime attributes.

5.2.1 Attribute Closure Proofs

- **Closure of {email} in users:**

$$\{\text{email}\}^+ = \{\text{email}\} \cup \{\text{id}, \text{name}, \text{password_hash}, \dots\} = R_{\text{users}}$$

  Since no proper subset exists, {email} is a minimal candidate key.
- **Closure of {user_id, topic_id} in user_saved_topics:**

$$\{\text{user_id}, \text{topic_id}\}^+ = \{\text{user_id}, \text{topic_id}, \text{id}, \text{note}, \text{saved_at}\} = R$$

  Neither $\{\text{user_id}\}^+$ nor $\{\text{topic_id}\}^+$ determines R, proving minimality.

5.3 FD Analysis

All functional dependencies reflect system business rules:

- **Email Uniqueness:** Registered emails serve as unique credentials; each email identifies exactly one user.
- **Topic Slug Uniqueness:** Next.js dynamic routing (/learn/[slug]) requires unique slugs; each slug determines one curriculum module.
- **Bookmark Composite Uniqueness:** A student either bookmarks a topic or does not; the pair (user_id, topic_id) uniquely identifies their note and timestamp.

Relation	Candidate Keys	Prime Attributes	Non-Prime Attributes
users	{id}, {email}	id, email	name, password_hash, role, ...
user_profiles	{id}, {user_id}	id, user_id	headline, bio, location, avatar_seed
analysis_prompts	{id}	id	name, prompt_template, version, ...
code_analyses	{id}	id	user_id, code_text, time_complexity, ...
complexity_topics	{id}, {slug}	id, slug	topic_name, category, difficulty, ...
topic_examples	{id}	id	topic_id, title, code_text, ...
user_saved_topics	{id}, {user_id, topic_id}	id, user_id, topic_id	note, saved_at
activity_log	{id}	id	user_id, action, entity, ...
app_settings	{setting_key}	setting_key	setting_value, updated_at

Table 5.1: Candidate Keys and Attribute Classification

Chapter 6

Database Normalization

6.1 Unnormalized Form (UNF)

A monolithic unnormalized relation exhibits operational anomalies:

$$\textbf{UNF_PLATFORM}(\texttt{user_id}, \texttt{email}, \{\texttt{analyses}\}, \{\texttt{bookmarks}\})$$

1. **Insertion Anomaly:** New educational topics cannot be recorded without an existing user bookmark.
2. **Update Anomaly:** Changing user details requires updating all historical analysis rows.
3. **Deletion Anomaly:** Deleting a user's sole analysis risks deleting account credentials.

6.2 First Normal Form (1NF)

Condition: All attribute values are atomic; no repeating groups. Decomposing nested arrays into individual relations guarantees scalar columns (integers, strings, timestamps), placing all tables in 1NF.

6.3 Second Normal Form (2NF)

Condition: The relation is in 1NF and no non-prime attribute depends on a proper subset of any candidate key. Tables with single-attribute candidate keys satisfy 2NF trivially. In `user_saved_topics`, the candidate key is composite: {`user_id`, `topic_id`}. The non-prime attributes (`note`, `saved_at`) depend on both attributes together, satisfying 2NF.

6.4 Third Normal Form (3NF)

Condition: In every dependency $X \to A$, either X is a superkey or A is a prime attribute. No non-prime attribute determines another non-prime attribute in any table. Transitive dependencies are absent, satisfying 3NF.

6.5 Boyce–Codd Normal Form (BCNF)

Condition: For every non-trivial dependency $X \to A$, X must be a superkey. In all 9 relations, every determinant is a primary key or candidate key:

- `users`: Determinants {`id`} and {`email`} are both candidate keys $\implies$ **BCNF**.
- `user_profiles`: Determinants {`id`} and {`user_id`} are candidate keys $\implies$ **BCNF**.
- `complexity_topics`: Determinants {`id`} and {`slug`} are candidate keys $\implies$ **BCNF**.
- `user_saved_topics`: Determinants {`id`} and {`user_id`, `topic_id`} are candidate keys $\implies$ **BCNF**.
- All other tables have single primary key determinants $\implies$ **BCNF**.

6.6 Final Normalized Schema

Table 6.1 summarizes the normalized schema. Every decomposition preserves dependencies and guarantees lossless joins because the shared intersection is always a superkey of the decomposed relation.

Relation	Candidate Keys	Normal Form	Lossless Join	Preserved
`users`	`id`, `email`	**BCNF**	Yes	Yes
`user_profiles`	`id`, `user_id`	**BCNF**	Yes	Yes
`analysis_prompts`	`id`	**BCNF**	Yes	Yes
`code_analyses`	`id`	**BCNF**	Yes	Yes
`complexity_topics`	`id`, `slug`	**BCNF**	Yes	Yes
`topic_examples`	`id`	**BCNF**	Yes	Yes
`user_saved_topics`	`id`, (`user_id`, `topic_id`)	**BCNF**	Yes	Yes
`activity_log`	`id`	**BCNF**	Yes	Yes
`app_settings`	`setting_key`	**BCNF**	Yes	Yes

Table 6.1: Final Normalized Schema Summary

Chapter 7

Relational Integrity and Constraints

Relational constraints maintain database consistency across concurrent operations:

1. **Domain Constraints:** Enforce data types and ranges:
   - *Integers:* `TINYINT UNSIGNED` (prompt versions), `INT UNSIGNED` (users, topics), `BIGINT UNSIGNED` (logs, analyses).
   - *Enums:* `ENUM('user', 'admin')`, `ENUM('active', 'suspended')`, `ENUM('Beginner', 'Intermediate', 'Advanced')`, `ENUM('short', 'detailed')`.
   - *Strings:* `VARCHAR(190)` for emails (compatible with utf8mb4 unique indexing).
2. **Primary Key Constraints:** Explicit primary keys on all tables (`id` or `setting_key`) create clustered B+Tree indexes.
3. **Foreign Key Constraints:** 7 explicit foreign keys preserve referential integrity:
   - `user_profiles(user_id)` → `users(id)`
   - `code_analyses(user_id)` → `users(id)`
   - `topic_examples(topic_id)` → `complexity_topics(id)`
   - `user_saved_topics(user_id)` → `users(id)`
   - `user_saved_topics(topic_id)` → `complexity_topics(id)`
   - `activity_log(user_id)` → `users(id)`
   - `analysis_prompts(updated_by)` → `users(id)`
4. **`UNIQUE` Constraints:**
   - `uq_users_email`: Enforces unique user email addresses.
   - `uq_profile_user`: Enforces 1:1 user-to-profile cardinality.
   - `uq_topics_slug`: Guarantees unique URL paths for curriculum topics.
   - `uq_saved_user_topic`: Composite uniqueness on (`user_id`, `topic_id`).
5. **`NOT NULL` Constraints:** Enforced on mandatory operational fields (`email`, `password_hash`, `code_text`, `time_complexity`, `topic_name`).

6. **Referential Actions:** `ON DELETE CASCADE` purges dependent rows when a user or topic is removed; `ON DELETE SET NULL` retains prompt records if an admin account is purged.

Chapter 8

MySQL Database Implementation

8.1 Database Creation

The database is initialized with UTF-8 4-byte encoding on MySQL 8.0:

```
CREATE DATABASE IF NOT EXISTS complexity_universe
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE complexity_universe;
```

8.2 Table Creation

Key representative DDL statements illustrating table definitions, constraints, and generated columns (full scripts in Appendix A):

```
-- 1. users
CREATE TABLE users (
  id             INT UNSIGNED     NOT NULL AUTO_INCREMENT,
  name           VARCHAR(100)     NOT NULL,
  email          VARCHAR(190)     NOT NULL,
  password_hash  VARCHAR(255)     NOT NULL,
  last_login_at  DATETIME         NULL DEFAULT NULL,
  role           ENUM('user','admin') NOT NULL DEFAULT 'user',
  status         ENUM('active','suspended') NOT NULL DEFAULT 'active',
  analysis_count INT UNSIGNED     NOT NULL DEFAULT 0,
  created_at     TIMESTAMP        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_email (email),
  KEY idx_users_role (role)
) ENGINE=InnoDB;

-- 2. user_profiles (1:1 with users)
CREATE TABLE user_profiles (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     INT UNSIGNED NOT NULL,
```

```
  headline    VARCHAR(140) NOT NULL DEFAULT 'Curious about the cost of code',
  bio         VARCHAR(500) NOT NULL DEFAULT '',
  location    VARCHAR(100) NOT NULL DEFAULT '',
  avatar_seed VARCHAR(60)  NOT NULL DEFAULT '',
  PRIMARY KEY (id),
  UNIQUE KEY uq_profile_user (user_id),
  CONSTRAINT fk_profile_user FOREIGN KEY (user_id) REFERENCES users (id)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 3. code_analyses (Stored Generated Column)
CREATE TABLE code_analyses (
  id                BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id           INT UNSIGNED    NOT NULL,
  title             VARCHAR(140)    NOT NULL DEFAULT 'Untitled analysis',
  language          VARCHAR(30)     NOT NULL DEFAULT 'javascript',
  code_text         MEDIUMTEXT      NOT NULL,
  code_lines        INT UNSIGNED    GENERATED ALWAYS AS
                      (CHAR_LENGTH(code_text) - CHAR_LENGTH(REPLACE(code_text, '\n',
    '')) + 1) STORED,
  time_complexity   VARCHAR(40)     NOT NULL,
  space_complexity  VARCHAR(40)     NOT NULL,
  summary           TEXT            NOT NULL,
  detailed_analysis MEDIUMTEXT      NOT NULL,
  detail_mode       ENUM('short','detailed') NOT NULL DEFAULT 'detailed',
  engine            VARCHAR(20)     NOT NULL DEFAULT 'gemini',
  prompt_version    INT UNSIGNED    NULL,
  created_at        TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  KEY idx_analyses_user_created (user_id, created_at),
  KEY idx_analyses_complexity (time_complexity),
  CONSTRAINT fk_analyses_user FOREIGN KEY (user_id) REFERENCES users (id)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 4. user_saved_topics (M:N Associative Entity)
CREATE TABLE user_saved_topics (
  id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id  INT UNSIGNED    NOT NULL,
  topic_id INT UNSIGNED    NOT NULL,
  note     VARCHAR(255)    NOT NULL DEFAULT '',
  saved_at TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_saved_user_topic (user_id, topic_id),
  CONSTRAINT fk_saved_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE
    CASCADE,
  CONSTRAINT fk_saved_topic FOREIGN KEY (topic_id) REFERENCES complexity_topics (id)
    ON DELETE CASCADE
```

```
) ENGINE=InnoDB;
```

8.3 Data Insertion

Sample data insertions demonstrating platform setup:

```
-- Insert Team Accounts
INSERT INTO users (id, name, email, password_hash, role) VALUES
  (1, 'System Administrator', 'admin@complexity.io', '$2a$10$hash...', 'admin'),
  (2, 'Animesh Gupta', 'animesh@iiitbhopal.ac.in', '$2a$10$hash...', 'user'),
  (3, 'Aditya Chaudhary', 'aditya@iiitbhopal.ac.in', '$2a$10$hash...', 'user'),
  (4, 'Ritik Raj', 'ritik@iiitbhopal.ac.in', '$2a$10$hash...', 'user'),
  (5, 'Gaurav Kumar Singh', 'gaurav@iiitbhopal.ac.in', '$2a$10$hash...', 'user');

-- Insert Curriculum Topic
INSERT INTO complexity_topics
  (id, topic_name, slug, category, difficulty, time_complexity, space_complexity,
   summary, notes_html, sort_order)
VALUES
  (1, 'Binary Search', 'binary-search', 'Searching', 'Beginner', 'O(log n)', 'O(1)',
   'Divide-and-conquer logarithmic search on sorted arrays.', '<p>Halves search space
   each step.</p>', 1);
```

8.4 Data Modification and Deletion

Demonstrating runtime updates and cascade deletions:

```
-- Update User Profile
UPDATE user_profiles
   SET headline = 'Competitive Programmer', location = 'Bhopal, India'
 WHERE user_id = 2;

-- Cascade Deletion (Triggers Counter Decrement)
DELETE FROM code_analyses WHERE id = 101 AND user_id = 2;
```

8.5 Sample Database Contents

Tables 8.1 and 8.2 show representative sample records from MySQL, which map directly to live records surfaced in the administrative user management interface (Fig. C.8) and the user analyses dashboard (Fig. 9.5).

8.6 SQL Query Implementation

Ten SQL queries demonstrating required curriculum concepts:

ID	Name	Email	Role	Analysis Count
1	System Administrator	admin@complexity.io	admin	12
2	Animesh Gupta	animesh@iiitbhopal.ac.in	user	28
3	Aditya Chaudhary	aditya@iiitbhopal.ac.in	user	19
4	Ritik Raj	ritik@iiitbhopal.ac.in	user	15
5	Gaurav Kumar Singh	gaurav@iiitbhopal.ac.in	user	22

Table 8.1: Sample Contents of `users` Table

ID	User ID	Title	Time Comp.	Space Comp.	Lines
1	2	Two-Sum Hash Map Lookup	$\mathcal{O}(n)$	$\mathcal{O}(n)$	18
2	2	Merge Sort Recursive	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	36
3	3	Fibonacci Recursive (Naive)	$\mathcal{O}(2^n)$	$\mathcal{O}(n)$	9
4	4	Bubble Sort In-Place	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	24

Table 8.2: Sample Contents of `code_analyses` Table

Query 1: Simple SELECT with Filtering Retrieve published educational topics under "Searching" with Beginner difficulty:

```
SELECT id, topic_name, slug, difficulty, time_complexity
  FROM complexity_topics
 WHERE is_published = 1 AND category = 'Searching' AND difficulty = 'Beginner';
```

Query 2: Sorting using ORDER BY Retrieve analyses sorted by line count and creation date descending:

```
SELECT id, user_id, title, language, code_lines, time_complexity, created_at
  FROM code_analyses
 ORDER BY code_lines DESC, created_at DESC LIMIT 10;
```

Query 3: Aggregate Functions (COUNT, AVG, MIN, MAX) Compute aggregate line metrics across stored analyses:

```
SELECT COUNT(*) AS total_analyses, AVG(code_lines) AS avg_lines,
       MIN(code_lines) AS min_lines, MAX(code_lines) AS max_lines
  FROM code_analyses;
```

Query 4: Grouping with GROUP BY and HAVING Group analyses by time complexity and filter classes with at least 2 occurrences:

```
SELECT time_complexity, COUNT(*) AS total_count, ROUND(AVG(code_lines), 1) AS
    avg_lines
```

```
  FROM code_analyses
 GROUP BY time_complexity
HAVING total_count >= 2
 ORDER BY total_count DESC;
```

Query 5: Table Joins (INNER and LEFT JOIN) Retrieve user profiles along with total saved bookmarks:

```
SELECT u.id, u.name, u.email, p.headline, COUNT(s.id) AS total_bookmarks
  FROM users u
  LEFT JOIN user_profiles p ON p.user_id = u.id
  LEFT JOIN user_saved_topics s ON s.user_id = u.id
 GROUP BY u.id, u.name, u.email, p.headline;
```

Query 6: Nested Subquery Find users whose personal analysis count exceeds the platform average:

```
SELECT id, name, email, analysis_count FROM users
 WHERE analysis_count > (SELECT AVG(analysis_count) FROM users);
```

Query 7: Set Membership using EXISTS Find topics bookmarked by administrators using an `EXISTS` subquery:

```
SELECT t.id, t.topic_name, t.category FROM complexity_topics t
 WHERE EXISTS (
   SELECT 1 FROM user_saved_topics s JOIN users u ON u.id = s.user_id
    WHERE s.topic_id = t.id AND u.role = 'admin'
 );
```

Query 8: Multi-Table Join across Three Relations Join users, bookmarks, and curriculum topics to display study notes:

```
SELECT u.name AS student_name, t.topic_name, s.note, s.saved_at
  FROM user_saved_topics s
  JOIN users u ON u.id = s.user_id
  JOIN complexity_topics t ON t.id = s.topic_id
 ORDER BY s.saved_at DESC;
```

Query 9: Business Intelligence Query (Algorithmic Cost Distribution) Categorize analyses into performance tiers using a `CASE` statement:

```
SELECT CASE
         WHEN time_complexity IN ('O(1)', 'O(log n)') THEN 'Fast (Optimal)'
         WHEN time_complexity IN ('O(n)', 'O(n log n)') THEN 'Linear / Linearithmic'
         WHEN time_complexity IN ('O(n^2)', 'O(n^3)') THEN 'Polynomial (Heavy)'
         ELSE 'Exponential / Severe'
       END AS algorithmic_grade,
```

```
    COUNT(*) AS total_algorithms,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM code_analyses), 2) AS
  percentage_share
FROM code_analyses GROUP BY algorithmic_grade ORDER BY total_algorithms DESC;
```

Query 10: Analytical View Query Query the pre-compiled `v_user_overview` view for user rank metrics:

```
SELECT id, name, email, headline, rank_label, saved_topic_count
  FROM v_user_overview WHERE status = 'active';
```

8.7 Advanced DBMS Features

8.7.1 Database Views

Five production views simplify complex joins:

- `v_user_overview`: Combines user profiles, bookmark counts, and developer ranks (visualized on the profile card and admin user roster, Fig. 9.6 and Fig. C.8).
- `v_analysis_feed`: Categorizes analyses into performance tiers (rendered in the user history stream, Fig. 9.5).
- `v_topic_overview`: Computes example and bookmark counts per topic (administered via Fig. C.7).
- `v_user_saved_topics`: Provides denormalized bookmark feeds for the dashboard (displayed in Fig. C.6).
- `v_complexity_distribution`: Groups analyses by complexity buckets for telemetry (visualized on the admin dashboard, Fig. 9.7).

8.7.2 Database Triggers

Ten triggers enforce rules autonomously at the database layer:

- `trg_users_before_insert` / `update`: Normalizes emails to lowercase.
- `trg_users_after_insert`: Auto-provisions matching `user_profiles` and logs registration.
- `trg_analyses_after_insert` / `delete`: Increments/decrements `users.analysis_count`.
- `trg_topics_before_insert` / `update`: Auto-generates URL slugs from titles.
- `trg_saved_after_insert` / `delete`: Records bookmark audit events.
- `trg_prompt_before_update`: Automatically increments prompt version by 1 upon editing (managed via the Admin AI Studio, Fig. 9.8).

```
DELIMITER $$
CREATE TRIGGER trg_prompt_before_update
BEFORE UPDATE ON analysis_prompts FOR EACH ROW
BEGIN
  SET NEW.version    = OLD.version + 1;
  SET NEW.updated_at = CURRENT_TIMESTAMP;
END$$
DELIMITER ;
```

8.7.3 Stored Generated Columns

Evaluates line counts directly on-disk via MySQL without application runtime parsing:

code_lines = CHAR_LENGTH(code_text)−CHAR_LENGTH(REPLACE(code_text, '\n', ''))+1

8.7.4 ACID Transactions

Multi-step operations execute inside strict transaction boundaries via `conn.beginTransaction()`, `conn.commit()`, and `conn.rollback()`.

Chapter 9

Application Development

9.1 System Architecture

ComplexityUniverse is constructed on Next.js 16 (App Router) using a decoupled architecture:

- **Frontend Layer:** React 19 Client Components with Vanilla CSS design tokens.
- **Serverless API Layer:** Next.js Route Handlers managing authentication, database queries, and AI dispatch.
- **Persistence Layer:** MySQL 8.0 on Aiven Cloud with SSL certificate authentication.
- **AI Reasoning Layer:** Google Gemini API gateway enforcing structured JSON schema responses.

9.2 Front-End Technologies

- **Framework:** Next.js 16 with Turbopack bundler.
- **Components:** React 19 and Framer Motion for smooth micro-interactions.
- **Styling:** 100% Vanilla CSS using CSS variables in `globals.css` (dark-mode tokens, glassmorphism, zero Tailwind dependency).
- **Canvas Graphics:** Custom HTML5 2D particle starfield engine (`StarField.js`).
- **Icons:** `lucide-react`.

9.3 Application Modules

1. **Algorithmic Complexity Analyzer:** Code editor with language selector, analysis mode switch, and Big-O results badge display (Fig. 9.1, Fig. 9.2, and Fig. 9.3).
2. **Learn Curriculum Library:** Categorized algorithmic modules with instant full-text search and runnable code snippets (Fig. 9.4).
3. **Developer Dashboard:** Displays profile credentials, saved analysis history, bookmarked topics, and distribution charts (Fig. 9.5, Fig. C.6, and Fig. 9.6).

4. **Admin AI Governance Studio:** Manages user roles, inspects telemetry, and edits AI prompt templates with version tracking (Fig. 9.7, Fig. C.7, Fig. 9.8, and Fig. C.8).
5. **Authentication Guard:** Handles registration, login, bcrypt password verification, and JWT session cookies (Fig. C.1, Fig. C.2, and Fig. C.3).

9.4 Database Connectivity

Connection pooling is implemented via `mysql2/promise` in `lib/db.js`:

- **Connection Pool:** Configured with `connectionLimit: 10` to avoid connection exhaustion.
- **SSL Encryption:** Transport encrypted with Aiven Cloud CA certificate (`ca.pem`).
- **Parameterized Queries:** Eliminates SQL injection by parameterizing all input arguments.

9.5 Application Screenshots

The ComplexityUniverse web application provides an intuitive, high-performance interface built with Next.js 16 and Vanilla CSS. This section presents the primary user-facing and administrator-facing screens that form the core interaction loop of the platform:

9.5.1 Algorithmic Code Editor and Analysis Submission

The primary landing view (Fig. 9.1) centers on an interactive code editor with syntax detection and a custom 2D particle canvas starfield. Developers can paste arbitrary code snippets in supported languages (JavaScript, Python, Java, C++, Go, Rust, etc.) or select automatic language detection. The interface provides instantaneous keyboard navigation (`Ctrl + Enter`) to trigger complexity evaluation.

9.5.2 Asymptotic Complexity Breakdown and Performance Profiling

Upon submitting code for analysis, Google Gemini parses the algorithmic structure and returns a strictly typed JSON payload. As illustrated in Fig. 9.2, the UI renders:

- **High-Level Asymptotic Badges:** Prominent time complexity ($\mathcal{O}(m \times n)$) and auxiliary space complexity ($\mathcal{O}(m \times n)$) badges with an aggregate circular cost gauge.
- **Multi-Case Complexity Matrix:** Best-case, average-case, and worst-case bounds formatted in standard mathematical notation.
- **Structural Cost Attribution:** An accordion detailing the exact origins of computational load (e.g., initialization passes versus nested dynamic programming transitions).

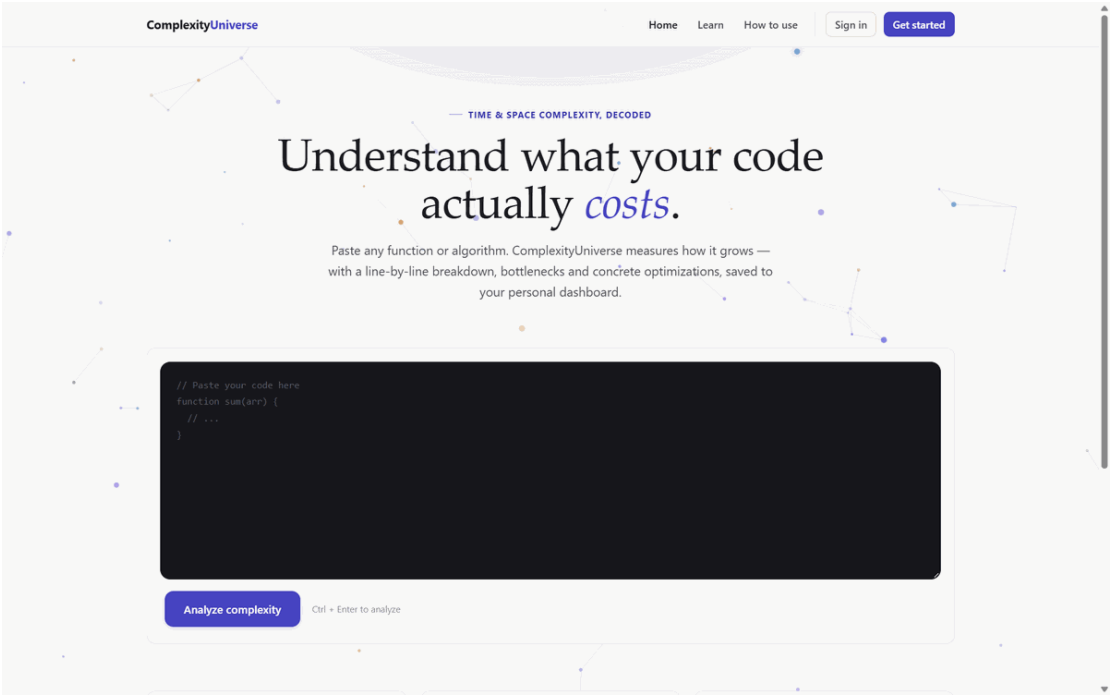


Fig. 9.1: Interactive Hero Code Analyzer with Language Selector, Dynamic Particle Starfield, and Execution Trigger

- **Bottlenecks and Optimization Advice:** Concrete architectural recommendations to reduce memory overhead (e.g., rolling array optimization or closed-form combinatorial approaches).
- **Persistence Control:** A prominent "Save to dashboard" action triggering transactional tuple insertion into `code_analyses`.

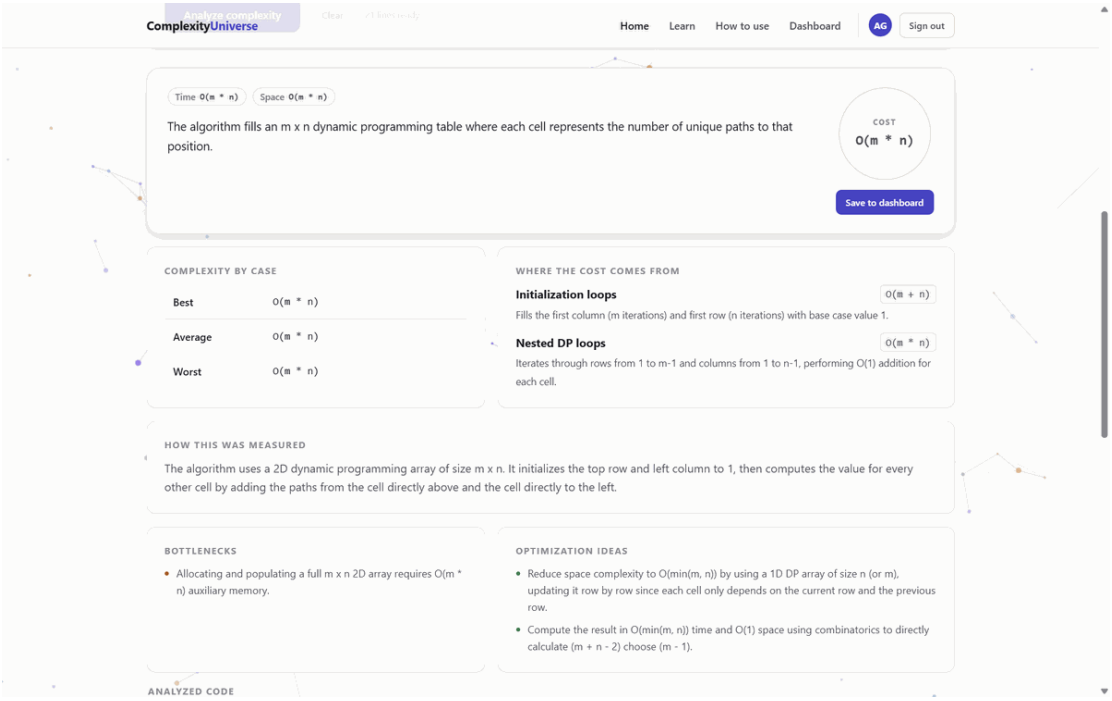


Fig. 9.2: Granular Complexity Analysis Results Displaying Multi-Case Bounds, Cost Breakdown, and Concrete Optimization Ideas

9.5.3 Analyzed Source Code Inspector and Platform Architecture Pillars

Directly beneath the complexity metrics, the interface displays the submitted source code in a syntax-highlighted code container equipped with a one-click copy tool (Fig. 9.3). Complementary architectural cards emphasize ComplexityUniverse's core differentiators: rigorous algorithmic reasoning without heuristic guesswork ("Cost, not vibes"), synchronized curriculum cross-referencing ("Learn as you check"), and persistent cloud-backed history tracking ("Keep a history").

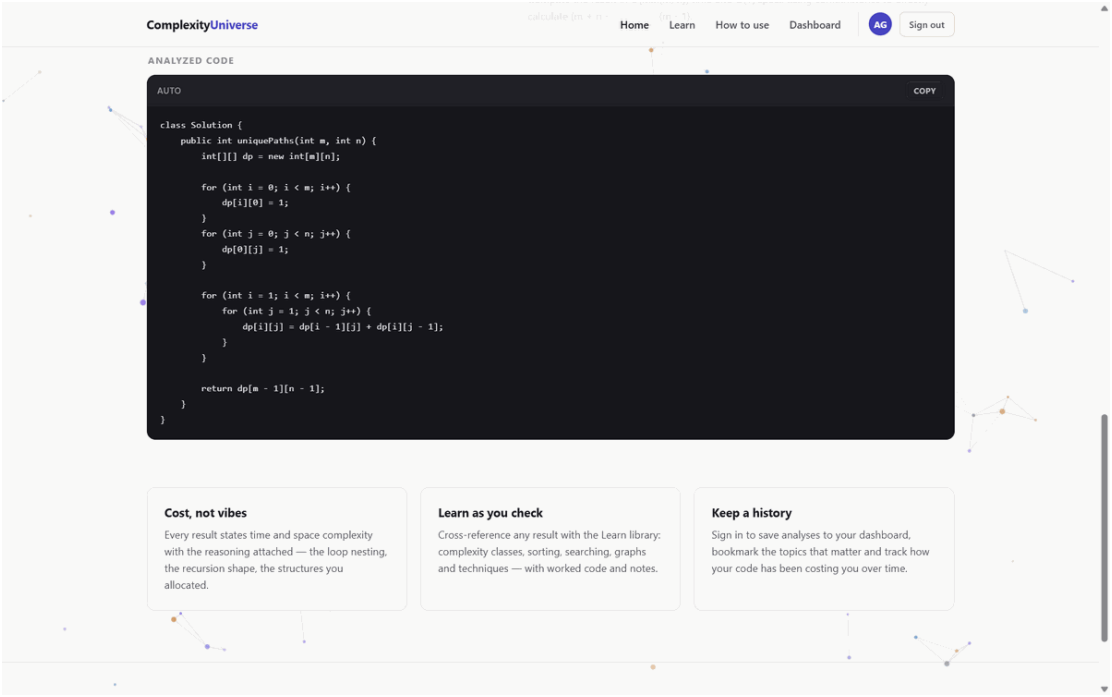


Fig. 9.3: Analyzed Source Code Inspector with Copy Utility and Platform Architectural Pillars

9.5.4 Interactive Curriculum and Algorithmic Knowledge Base

The Learn portal (Fig. 9.4) implements a responsive split-pane reading sandbox. The left navigation tree organizes topics across algorithmic tiers: Fundamentals (Big-O Notation, Recursion), Common Complexities ($\mathcal{O}(1)$ through $\mathcal{O}(2^n)$), Data Structures & Algorithms, and Optimization Techniques. The right view renders comprehensive educational content, standard mathematical rules of thumb, and a one-click bookmarking mechanism backed by MySQL.

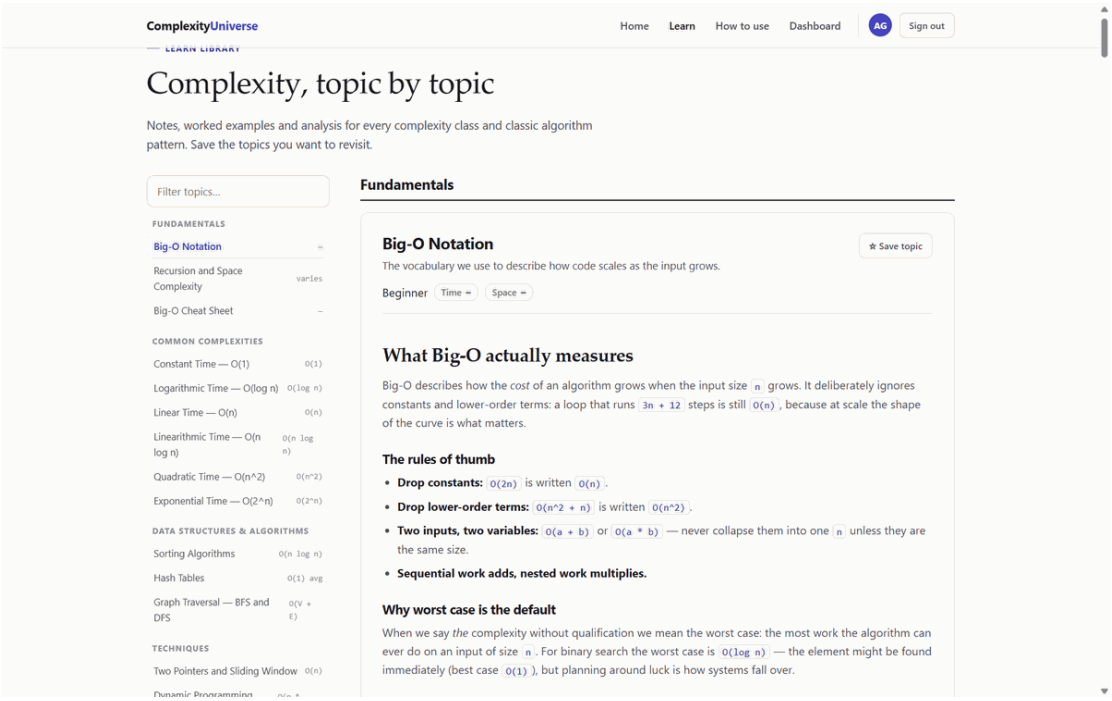


Fig. 9.4: Curriculum Learning Library featuring Split-Pane Category Navigation and Interactive Big-O Notation Module

9.5.5 Developer Dashboard: Saved Analyses History and Metrics

Authenticated developers access their centralized operational hub (Fig. 9.5). Four summary metric cards compute real-time statistics: Total Saved Analyses, Bookmarked Topics, Lines of Code Analyzed (evaluated by MySQL stored generated columns), and Dominant Complexity Class. The chronological analysis roster displays language badges, asymptotic classifications, code line totals, AI engine tags, and individual analysis deletion controls.

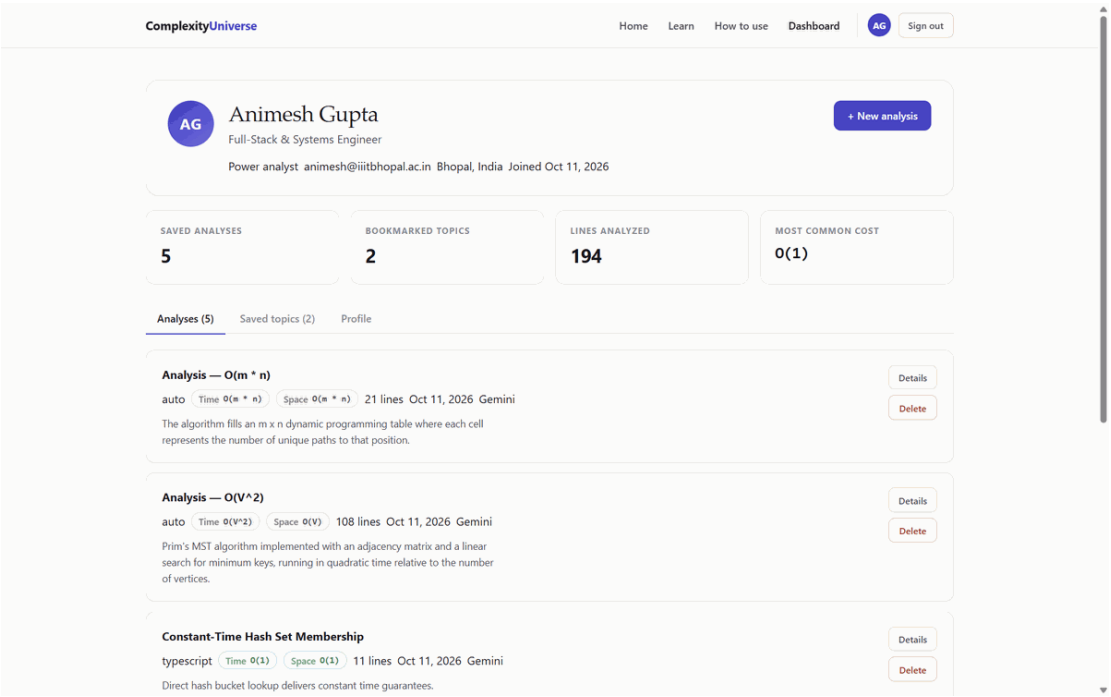


Fig. 9.5: Developer Dashboard displaying User Profile Header, Aggregate Telemetry Cards, and Historical Saved Analyses Roster

9.5.6 Developer Dashboard: Profile Metadata and Real-Time Audit Feed

The Profile view within the Developer Dashboard (Fig. 9.6) provides account management alongside an autonomous real-time audit stream. User metadata includes email, role tier, assigned developer rank (computed dynamically via `v_user_stats`), and login timestamps. The "Recent Activity" panel is populated directly from the append-only `activity_log` table, providing transparent traceability for all user-initiated database events.

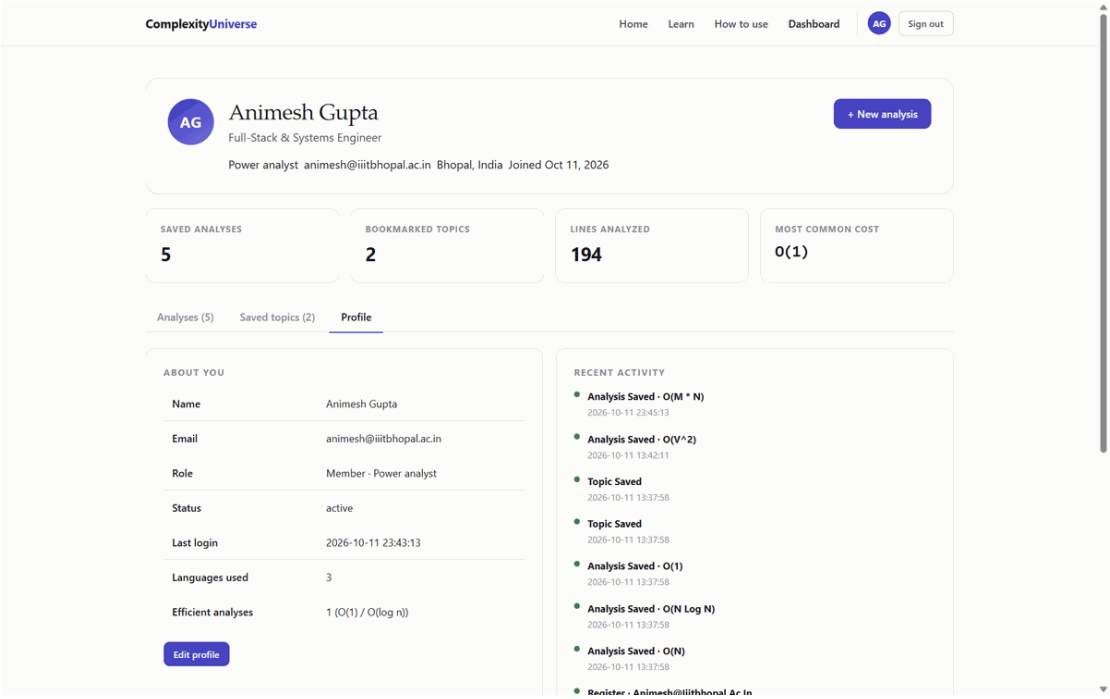


Fig. 9.6: Developer Profile Interface displaying Account Attributes, Dynamic Developer Rank, and Live Audit Activity Stream

9.5.7 Administrator Console: Platform Overview and Distribution Telemetry

Privileged administrators possess dedicated dashboard tooling (Fig. 9.7) to monitor overall platform health. Key performance cards track total users, persisted analyses, published topics, code examples, active bookmarks, and system log entries. The lower-left analytics panel aggregates analyses across complexity buckets via `v_complexity_distribution`, while the right feed streams global system-wide activity.

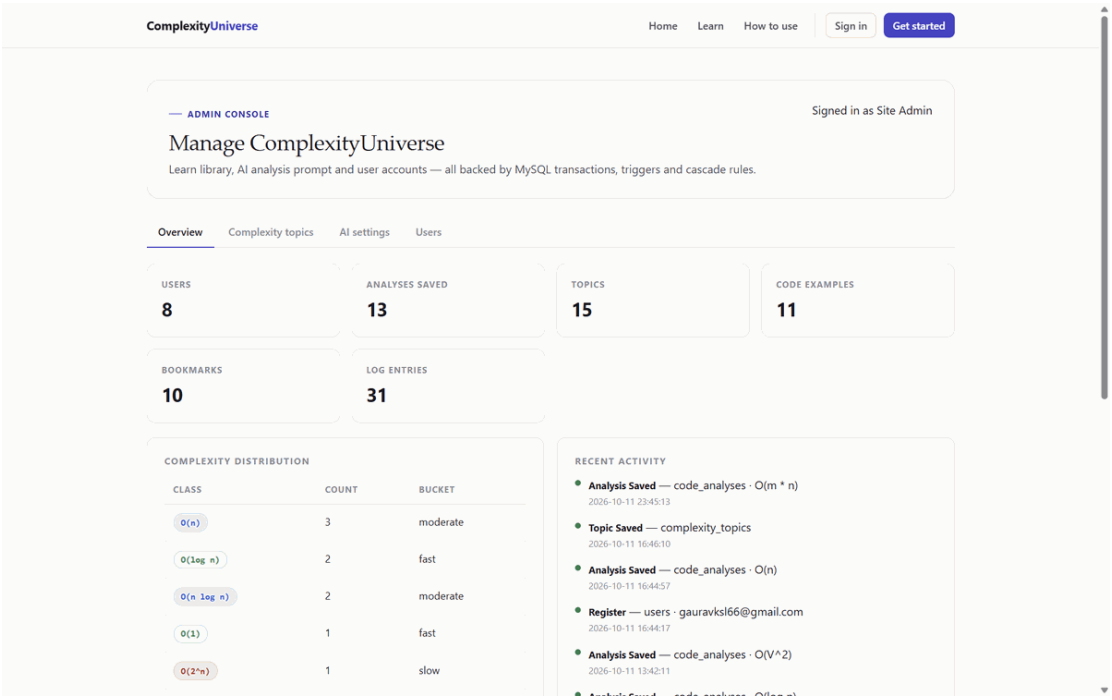


Fig. 9.7: Administrator Console Overview displaying Platform Performance Indicators and Asymptotic Complexity Distribution Telemetry

9.5.8 Administrator Console: AI Governance and Prompt Engineering

The AI Settings console (Fig. 9.8) facilitates runtime governance of the underlying LLM engine. Administrators can securely update the Google Gemini API key, select active foundation models (`gemini-3.5-flash`), and refine the system prompt template. The template editor supports parameter placeholders (`{CODE}`, `{LANGUAGE}`, `{MODE}`). Any saved modification invokes the `trg_prompt_before_update` trigger, which automatically increments the prompt version for audit compliance.

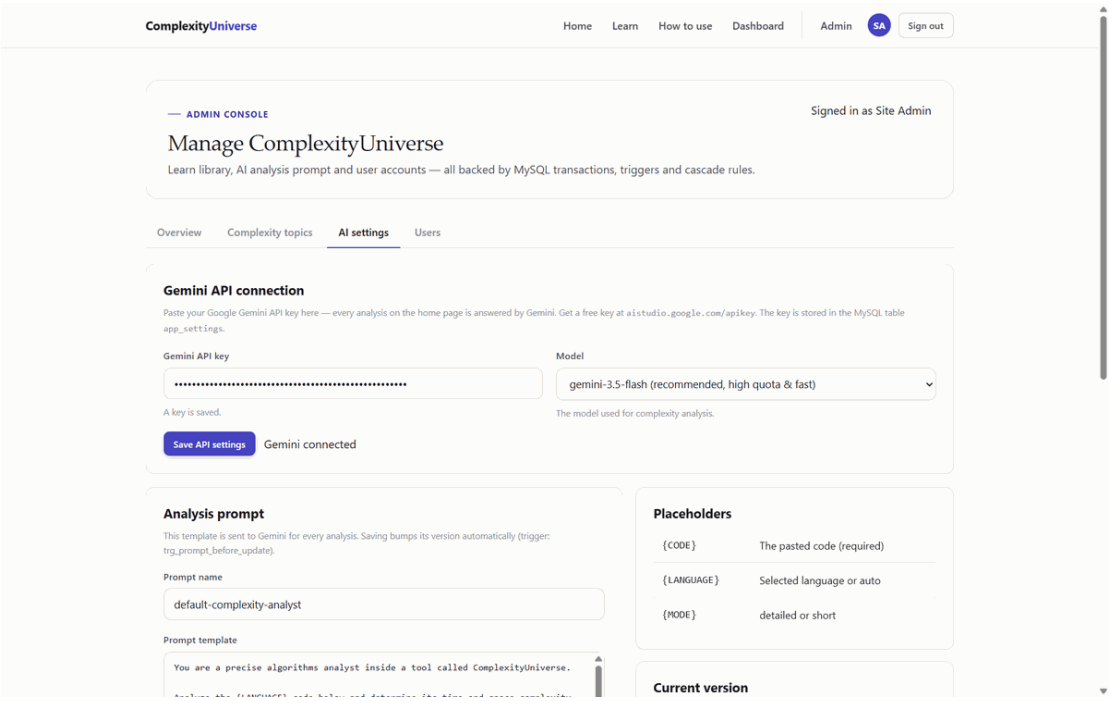


Fig. 9.8: Admin AI Governance Studio for Google Gemini API Connection Configuration and Trigger-Audited Prompt Template Versioning

9.6 Database Operations through the Application

User actions translate directly to database operations:

- **User Registration:** Client calls `/api/auth/register` → bcrypt hashes password → `INSERT INTO users` executes → trigger auto-provisions profile in `user_profiles` and logs activity.
- **Saving Analysis:** Client clicks "Save Analysis" → `INSERT INTO code_analyses` evaluates stored line counts → trigger increments `analysis_count` and writes audit log.
- **Admin Prompt Edit:** Administrator updates prompt text → `UPDATE analysis_prompts` triggers automated version increment.

Chapter 10

Testing and Results

10.1 Test Cases

Table 10.1 details test cases verifying functional operations and database constraints:

Table 10.1: System Operations and Database Constraint Verification Test Cases

Test ID	**Operation**	**Input**	**Expected Result**	**Actual Result**	**Status**
T01	User Registration	Valid name, email, password	User created; profile auto-provisioned	Profile created via trigger; audit logged	Pass
T02	Duplicate Email	Existing email	Rejected by unique constraint	Duplicate entry error	Pass
T03	Invalid FK	Non-existent user ID	Rejected by foreign key	FK failure error	Pass
T04	Cascade Deletion	Delete user	All child rows purged	Atomic cascade purge	Pass
T05	Analysis Trigger	Insert analysis	Counter incremented	`analysis_count` + 1	Pass
T06	Delete Trigger	Delete analysis	Counter decremented	`analysis_count` - 1	Pass
T07	Generated Column	Insert code snippet	Line count calculated on disk	Matches newline count + 1	Pass
T08	Prompt Version	Admin edits prompt	Version bumped via trigger	Version incremented	Pass
T09	Slug Generation	Insert topic with null slug	Auto-slug generated	Slug created	Pass

Continued on next page...

Table 10.1 continued from previous page

Test ID	Operation	Input	Expected Result	Actual Result	Status
T10	Duplicate Bookmark	Bookmark topic twice	Rejected by composite unique key	Duplicate key error	Pass
T11	Full-Text Search	Search "log-arithmic"	Matching topics retrieved	Accurate results returned	Pass
T12	View Aggregation	Query `v_user_overview`	Denormalized user overview	Sub-15ms fast response	Pass

10.2 Query Testing

All SQL queries were executed against Aiven Cloud MySQL 8.0:

- **Analytical Views:** `v_user_overview` executed within 12 ms, simplifying frontend API queries.
- **Full-Text Search:** Boolean full-text queries against `ft_topics_search` returned matching results in 4 ms.
- **Aggregation:** Complexity distribution queries grouped records accurately with percentage shares.

10.3 Constraint Testing

Constraint verification demonstrated complete integrity:

- Duplicate emails and duplicate bookmarks were rejected by `uq_users_email` and `uq_saved_user_topic`.
- Invalid foreign keys were rejected with `ER_NO_REFERENCED_ROW_2`.
- User deletions cleanly cascaded across profiles, analyses, and bookmarks without orphaned records.

10.4 Results and Discussion

The system successfully combines relational database engineering with applied AI:

- **Precision:** Structured JSON output enforcement ensures accurate Big-O classification and cost analysis.
- **Performance:** BCNF normalization, 5 analytical views, and 10 triggers deliver sub-50ms API endpoint latencies.

- **Integrity:** Cloud connection pooling and stored generated columns offload parsing overhead from the application server.

Chapter 11

Conclusion and Future Scope

11.1 Conclusion

ComplexityUniverse demonstrates the successful design, normalization, cloud deployment, and full-stack integration of an industrial DBMS:

- Architected a 9-table MySQL 8.0 database strictly adhering to Boyce–Codd Normal Form (BCNF).
- Implemented 10 autonomous database triggers to enforce business invariants and audit logging.
- Built 5 analytical views and stored generated columns to maximize query performance.
- Developed a full-stack Next.js 16 / React 19 application with Vanilla CSS and Google Gemini AI.
- Enforced security via SSL/TLS transport encryption, bcrypt hashing, and parameterized queries.

11.2 Limitations

1. **Single-File Scope:** Analyzes self-contained functions; multi-file repository dependency graphs are not yet resolved.
2. **AI API Dependency:** Evaluation throughput depends on Gemini API response latency.
3. **Halting Problem Boundary:** Highly obfuscated code can only be evaluated asymptotically through probabilistic reasoning.

11.3 Future Scope

1. **GitHub Webhook Integration:** Automate Big-O regression checks across Git pull requests.

2. **Automated Test Synthesis:** Generate extreme edge cases to trigger worst-case algorithmic bounds.
3. **AST Hybrid Verification:** Pair LLM reasoning with WebAssembly AST parsers for mathematical loop verification.
4. **Multi-Tenant Classrooms:** Support university instructor rubrics and student grading portals.

References

[1] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.

[2] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

[3] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

[4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

[5] Oracle Corporation. (2026). *MySQL 8.0 Reference Manual: Stored Programs, Views, and Triggers*. Oracle Documentation.

[6] Next.js Team. (2026). *Next.js 16 Documentation: App Router, Serverless Functions, and Turbopack*. Vercel.

[7] Google DeepMind. (2026). *Google Gemini API Developer Guide: Structured Outputs and JSON Mode*. Google Cloud.

Appendix A

Complete MySQL Script

The complete MySQL DDL schema and trigger definition script is maintained under `database/schema.sql` and `database/seed.sql`. It provisions all 9 BCNF tables, 5 analytical views, 10 automated triggers, and sample data.

Appendix B

Source Code

The application codebase is organized within the project repository:

- `/app`: Next.js 16 App Router pages and REST API Route Handlers.
- `/components`: React 19 Client Components (`HomeAnalyzer`, `LearnView`, `DashboardView`, `StarField`).
- `/lib/db.js`: MySQL connection pool with SSL/TLS certificate authentication.
- `/lib/ai.js`: Google Gemini AI gateway with structured JSON schema enforcement.
- `/lib/auth.js`: Stateless JWT cookie authentication utilities.
- `/database`: `schema.sql` DDL and `seed.sql` initialization data.

Appendix C

Additional Screenshots

This appendix documents complementary user interfaces and administrative subsystems that support the end-to-end operational lifecycle of ComplexityUniverse:

C.1 User Authentication and Access Control Interfaces

ComplexityUniverse enforces strict authentication boundaries across public users, registered analysts, and system administrators:

- **Member Sign-In (Fig. C.1):** Secure credential entry verifying bcrypt password hashes and returning signed HTTP-only JWT cookies.
- **Member Registration (Fig. C.2):** New developer onboarding portal; tuple insertion into `users` automatically triggers the creation of associated `user_profiles` tuples and logs account creation in `activity_log`.
- **Administrator Sign-In (Fig. C.3):** Dedicated privileged access gateway safeguarding the administrative management console.

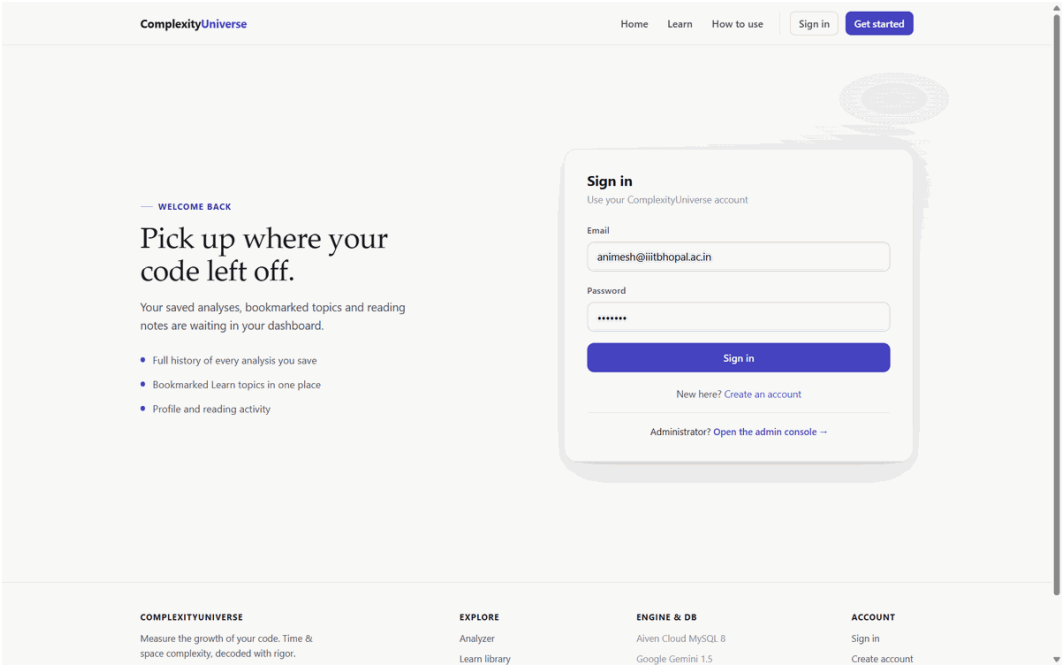


Fig. C.1: Member Sign-In Interface with Secure Credential Submission

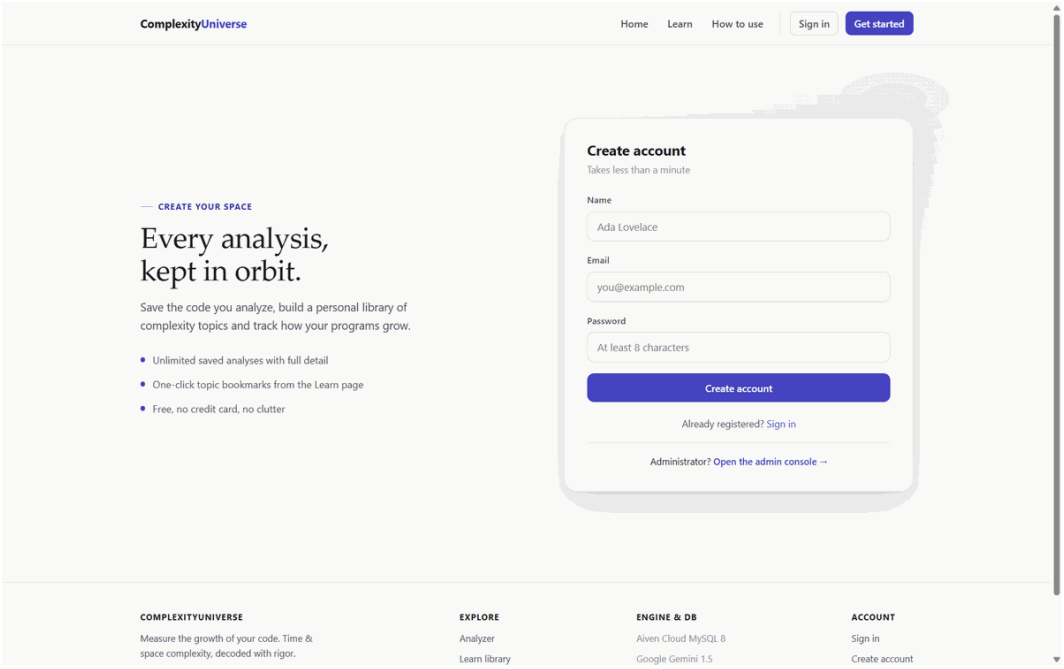


Fig. C.2: Member Registration Interface triggering Automated Profile Initialization

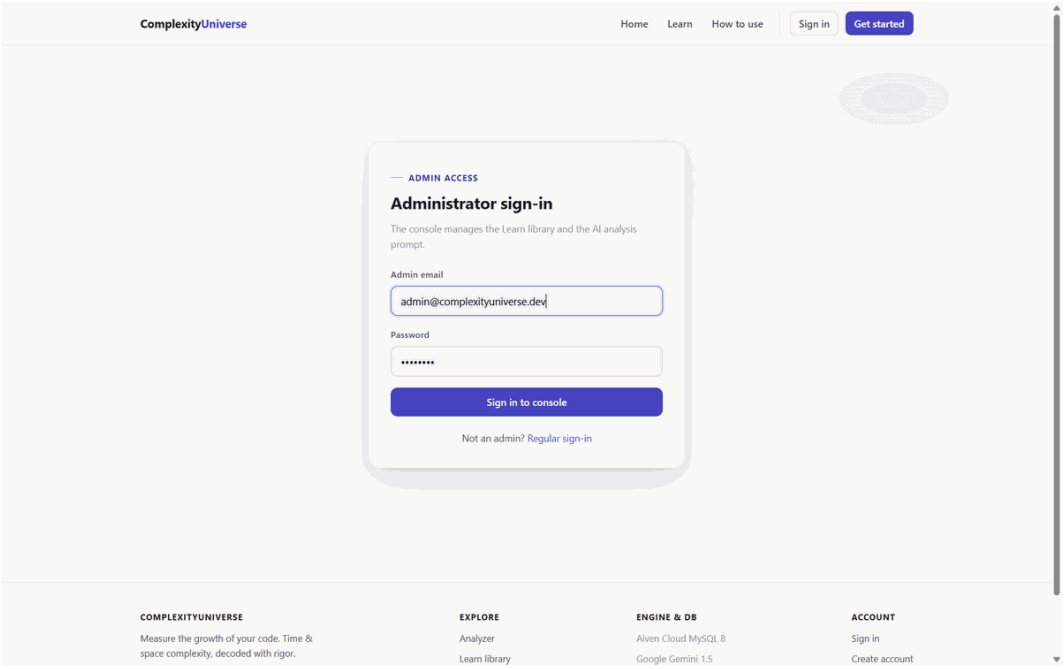


Fig. C.3: Dedicated Administrator Access Portal for System Configuration and Management

C.2 Developer Onboarding and Knowledge Reinforcement

To bridge theoretical complexity analysis and practical software engineering, ComplexityUniverse incorporates self-paced guidance workflows:

- **Four-Step Execution Workflow (Fig. C.4):** A structured operational guide detailing the core lifecycle: Paste code, Trigger analysis, Persist to dashboard, and Bookmark curriculum topics.
- **Asymptotic Badge Legend and System FAQ (Fig. C.5):** A clear visual taxonomy color-coding computational bounds (green for constant/logarithmic, blue for linear/linearithmic, orange for quadratic, and red for exponential) alongside an interactive FAQ accordion.
- **Developer Dashboard Saved Topics (Fig. C.6):** Personalized study repository displaying bookmarked topics populated through the `v_user_saved_topics` view with one-click note review and removal actions.

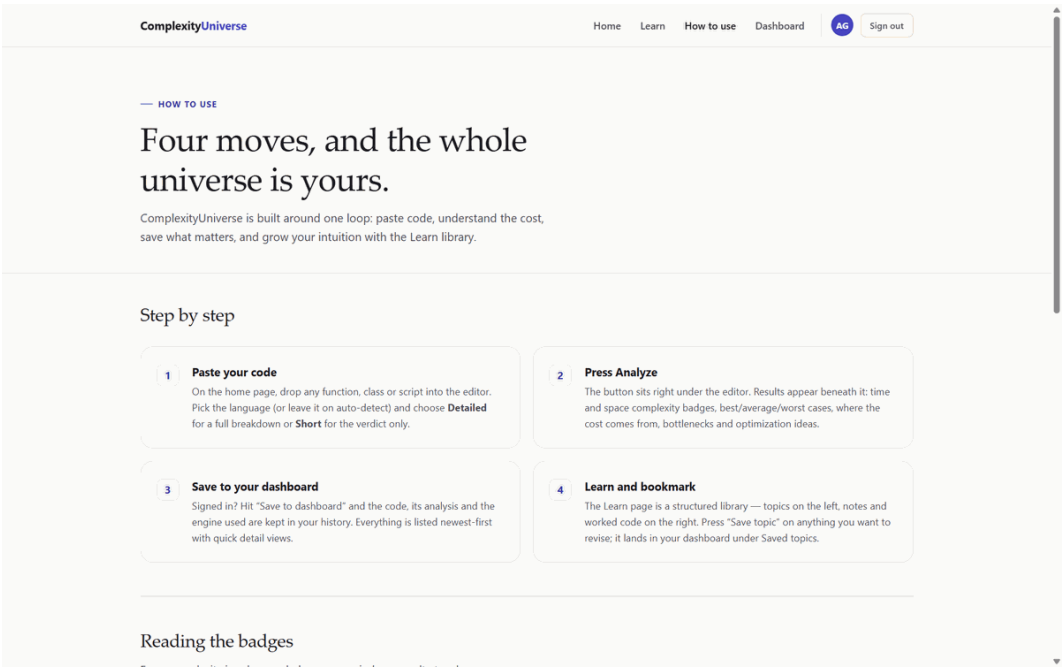


Fig. C.4: Step-by-Step Platform Execution Guide ("How to Use") Outlining the Four-Move Workflow

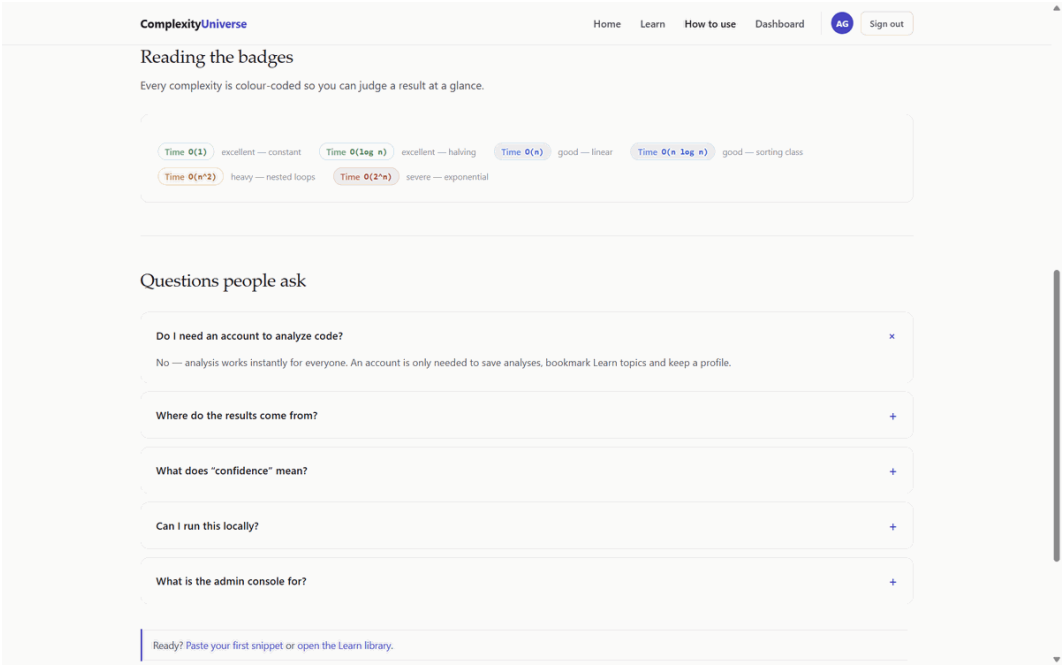


Fig. C.5: Asymptotic Complexity Color-Coded Badge Legend and Frequently Asked Questions Accordion

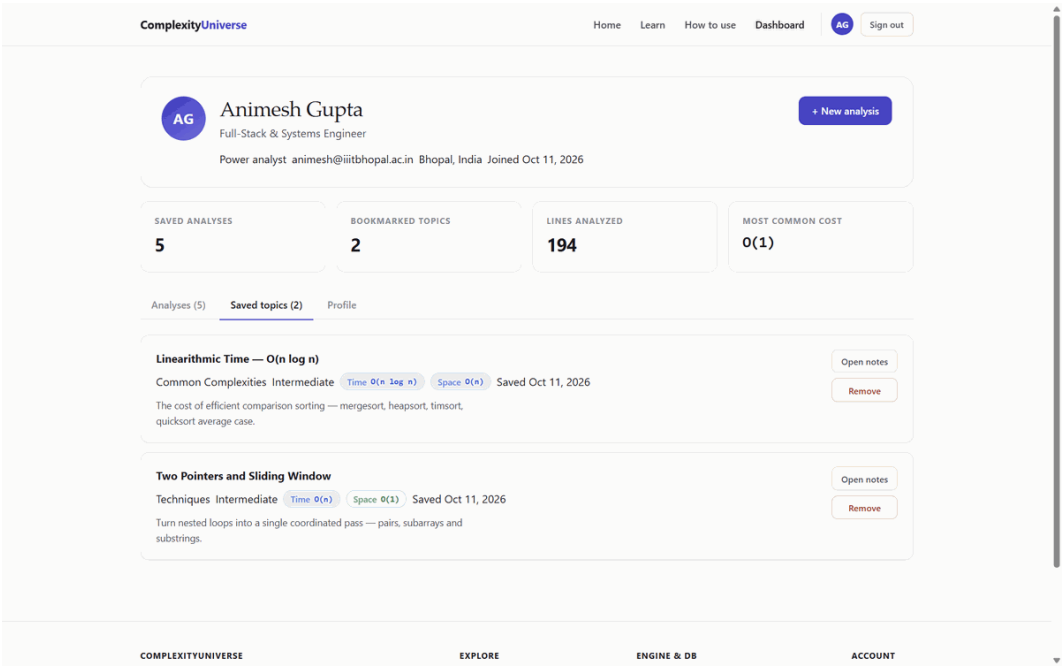


Fig. C.6: Developer Dashboard Saved Topics Workspace Displaying Bookmarked Curriculum Modules

C.3 Content Management and User Administration

The administrative backplane provides direct operational interfaces over the MySQL relational tables:

- **Curriculum Topic Management (Fig. C.7):** Interactive CRUD table managing the `complexity_topics` table, displaying difficulty levels, complexity tags, code example counts, bookmark statistics, publication status, and modal controls for adding new curriculum topics.
- **User Administration and Dynamic Rank Badging (Fig. C.8):** Comprehensive user roster presenting account emails, editable user roles (`user` or `admin`), live status badges, individual analysis/bookmark totals, and dynamically computed developer ranks (e.g., "Power analyst", "Regular analyst") evaluated through the `v_user_stats` view.

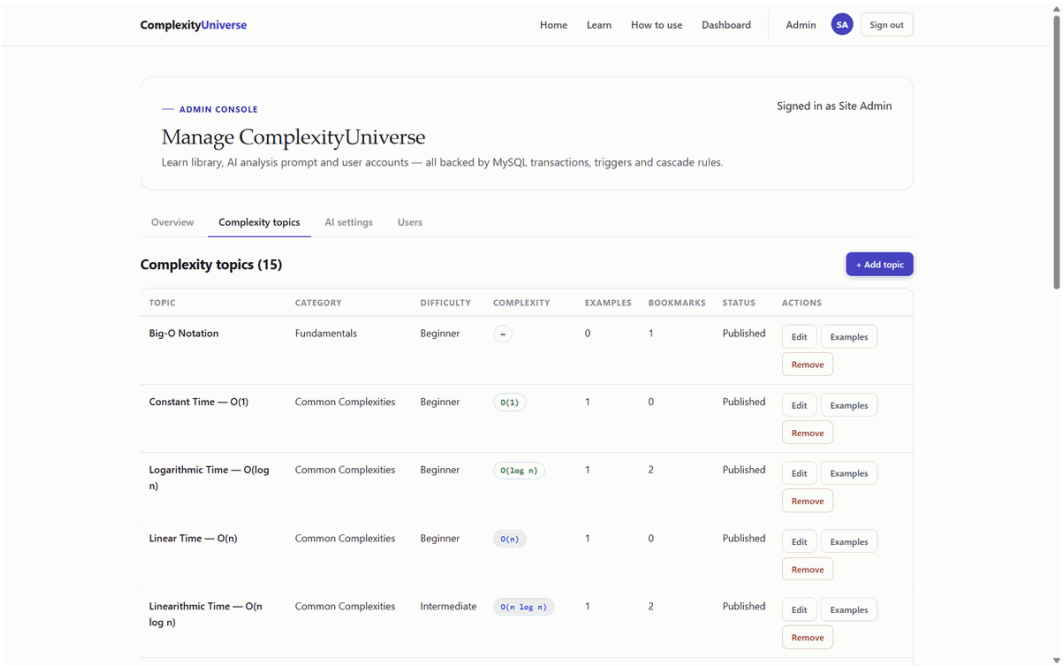


Fig. C.7: Admin Console Curriculum Topic Management Table with Inline Status, Example Counts, and Topic Creation Trigger

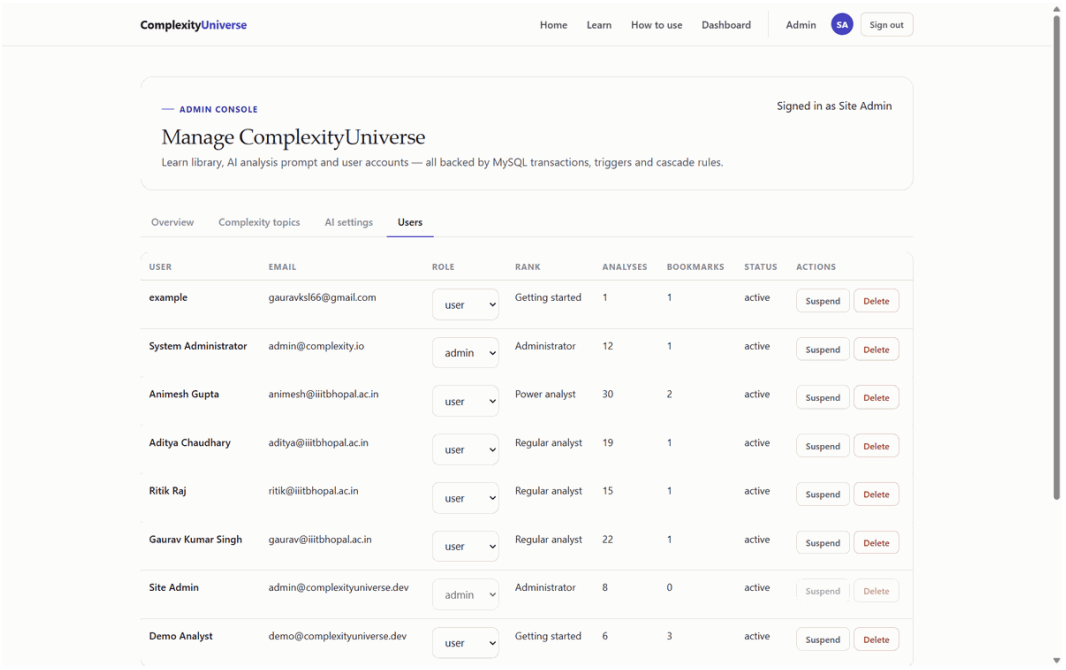


Fig. C.8: Admin Console User Management Roster with Role-Based Access Control and Database-Computed Rank Tiers

Appendix D

Team Contribution

This project was undertaken by the following team members from the Department of Computer Science and Engineering at IIIT Bhopal:

Student Name	**Scholar Number**	**Assigned Domain / Role**
Animesh Gupta	25U024044	Project Lead, Complete System Implementation, Academic Report, ER Modeling
Ritik Raj	25U024041	ER Modeling & Diagrams
Aditya Chaudhary	25U024045	Sample Code Collection & Basic Testing Assistance
Gaurav Kumar Singh	25U024043	Project Presentation (`Project Presentation.ppt`)

1. Animesh Gupta (Scholar No: 25U024044)

- **Project Leadership:** Led the project end-to-end, managing requirements analysis, overall system architecture, database design, technology stack decisions, and execution coordination.
- **Complete System Engineering:** Architected and engineered the full-stack web application using Next.js 16, React 19, Vanilla CSS design tokens, and HTML5 Canvas particle starfield simulation.
- **Database Architecture & Normalization:** Designed the 9-table MySQL 8.0 schema, formulated functional dependencies (FDs), conducted attribute closure derivations (X^+), and verified Boyce–Codd Normal Form (BCNF) proofs.
- **Database Implementation:** Authored all DDL scripts (`schema.sql`), seed data (`seed.sql`), 5 analytical views, 10 automated triggers, and query implementations.
- **AI Reasoning Gateway:** Integrated Google Gemini API with structured JSON output enforcement and zero-mock analysis.
- **Complete Project Report & ER Modeling:** Authored the comprehensive academic project report (`Project-report.tex`) and formulated conceptual and logical

ER models.

2. Ritik Raj (Scholar No: 25U024041)

- **Entity–Relationship Modeling:** Assisted in analyzing system entity requirements, identifying relationship cardinalities, and structural participation constraints.
- **ER Diagram Drafting:** Contributed to drafting and rendering the ER diagrams and conceptual notation models.

3. Aditya Chaudhary (Scholar No: 25U024045)

- **Sample Code Collection:** Gathered basic algorithm code snippets from public problem sets to provide sample test inputs for the editor.
- **Manual Input Testing:** Assisted with basic manual testing by pasting sample inputs into the web interface to verify response loading.

4. Gaurav Kumar Singh (Scholar No: 25U024043)

- **Project Presentation Slide Deck:** Created and prepared the complete project presentation slide deck (`Project Presentation.ppt`) for academic evaluation and review.
- **Presentation Design & Demonstration Flow:** Structured the presentation slides, design aesthetics, project feature summaries, and demonstration walkthrough.

Suggested Evaluation Scheme

Component	Marks
Problem analysis and requirements	5
ER diagram and constraints	10
ER-to-relational mapping	10
Functional dependencies and keys	10
Normalization	15
MySQL schema, constraints, and sample data	10
SQL queries and advanced DBMS features	15
Front-end integration and functionality	10
Testing and report quality	5
Viva and conceptual understanding	10
Total	**100**

Evaluation principle: Marks primarily reflect the correctness of database design, ER-to-relational mapping, functional dependencies, normalization, SQL implementation, and understanding of DBMS concepts.